

Integration Contracts

Every external system that plugs into the ISR C2 platform speaks against one of the contracts in this book. Sensors, NN classifiers, receiver agencies, dispatch APIs, IAM providers, agentic services, audit sinks — each has a documented shape any implementer can build against without asking us.

Read this before writing any adapter, integration, or customer-facing endpoint.

Version	Date	Source of truth
0.1 (Phase A initial)	2026-09-13	Working tree at HEAD

Structure

Every contract section follows the same shape:

1. **Purpose** — what integrates through this contract
2. **Actors** — who calls, who receives
3. **Wire format** — concrete schemas
4. **Auth model**
5. **Error semantics**
6. **Sequence diagram** — mermaid
7. **Example** — request / response
8. **Reference implementation** — pointer to mock adapter or stub in the codebase
9. **Open questions** — anything TBD

Table of contents

Phase A · Foundation contracts

- [1. Sensor ingestion](#) — RF / acoustic / visual / radar → platform
- [2. NN classifier](#) — raw detection → classified event
- [3. Site definition](#) — customer site schema + onboarding
- [4. Post-Incident Report export](#) — JSON / PDF / CSV bundling

Phase B · Integration contracts

- [5. Receiver adapter](#) — outbound to any of 386 registered receivers
- [6. Dispatch adapter](#) — outbound to real asset dispatch APIs
- [7. Server-side visibility](#) — the auth-layer gate `visibility.js` documents but doesn't implement

Phase C · Advanced contracts

- [8. Agent B \(Mistral\)](#) — narrative synthesis
- [9. Signature bridge](#) — raw NN signature → family label + candidate models
- [10. Feedback log WORM](#) — Azure Blob write-once audit sink
- [11. Cooperative traffic fusion](#) — ADS-B / Naviar reconciliation
- [12. Auth + IAM](#) — tenant provisioning + session management

Cross-cutting conventions

Applies to every contract in this book.

- **Encoding.** UTF-8 throughout. Never Latin-1. Danish characters (æ, ø, å) always preserved.
- **Time.** ISO 8601 with explicit `Z` suffix (UTC). Sub-second precision at millisecond resolution when ordering matters.
- **Coordinates.** WGS 84 latitude/longitude in decimal degrees. Altitude in metres above ellipsoid (WGS 84), NOT above ground, NOT above sea level. All calculations round-trip through Cesium's Cartographic type.

- **Identifiers.** Kebab-case strings for machine ids (`shahed-136` , `politi-kbh` , `evt-1234`). Freeform strings for human labels. Never mix.
- **Auth.** Every contract uses OAuth 2.0 client-credentials against the Auth+IAM contract (Section 12) unless explicitly noted. Tokens carry tenant + role claims.
- **Idempotency.** Every write endpoint accepts an `Idempotency-Key` header. Repeat requests with the same key + same payload return the original response. Repeat with same key + different payload returns HTTP 409.
- **Error format.** All 4xx / 5xx responses use RFC 7807 Problem Details JSON:

```
{
  "type": "https://isr-systems.dk/errors/invalid-signature",
  "title": "Signature payload rejected",
  "status": 400,
  "detail": "acoustic.confidence must be in [0.0, 1.0]",
  "instance": "/sensors/cph-radar-01/detections",
  "correlation-id": "abc-123"
}
```

- **Correlation.** Every request must carry an `X-Correlation-ID` header (UUID v4). Propagates through every downstream call. Surfaces in audit log + feedback log.
- **Rate limits.** Per-tenant, per-endpoint. Enforced at the API layer. `429 Too Many Requests` includes `Retry-After` header in seconds.
- **Versioning.** Every endpoint is versioned in the path: `/v1/sensors/...` . Breaking changes bump the major version + run parallel for one deprecation cycle.
- **Detection-only.** No contract in this book triggers a real-world dispatch action without a human confirming through the Mission Console. Every dispatch adapter (Section 6) requires a signed operator confirmation payload — the raw asset-command endpoint doesn't exist.

1. Sensor ingestion

Purpose

Every physical sensor (RF spectrum analyser, acoustic array, EO/IR camera, radar) posts detections into the platform through this contract. This is the entry point for real-world signal.

Actors

- **Caller:** sensor device or edge gateway on the customer's site
- **Receiver:** ISR C2 sensor ingest API (Azure Function endpoint per site tenant)

Wire format

POST `/v1/sensors/{sensor_id}/detections`

Batch endpoint. Sensors send one array of detections per second (or per fixed window). Batching amortises TLS handshake + reduces per-detection overhead.

Request body:

```
{
  "sensor_id": "cph-rf-01",
  "batch_id": "batch-1726220000-001",
  "window_start": "2026-09-13T10:30:00.000Z",
  "window_end": "2026-09-13T10:30:01.000Z",
  "detections": [
```

```

{
  "detection_id": "det-1726220000-abc",
  "timestamp": "2026-09-13T10:30:00.412Z",
  "modality": "rf",
  "position": {
    "lat": 55.617,
    "lon": 12.646,
    "alt_m": 120.5,
    "accuracy_m": 15.0
  },
  "signature": {
    "rf": {
      "band": "2.4-GHz",
      "center_hz": 2437000000,
      "bandwidth_hz": 20000000,
      "power_dbm": -68.2,
      "modulation_hint": "ofdm",
      "hopping_pattern": null
    }
  },
  "confidence": 0.87,
  "raw_reference": "s3://cph-rf-01-raw/2026-09-13/10-30-00-412.iq"
},
{
  "detection_id": "det-1726220000-def",
  "timestamp": "2026-09-13T10:30:00.520Z",
  "modality": "acoustic",
  "position": { "lat": 55.618, "lon": 12.647, "alt_m": null, "accuracy_m": 30.0 },
  "signature": {
    "acoustic": {
      "profile": "moped-buzz",
      "fundamental_hz": 78.0,
      "harmonic_ratio": 0.42,
      "spl_db": 54.3
    }
  },
  "confidence": 0.91,
  "raw_reference": "s3://cph-acoustic-01-raw/2026-09-13/10-30-00-520.wav"
}
]
}

```

Response 202 Accepted:

```

{
  "batch_id": "batch-1726220000-001",
  "accepted_count": 2,
  "rejected_count": 0,
  "rejections": [],
  "next_batch_after": "2026-09-13T10:30:01.000Z"
}

```

Field constraints:

- `sensor_id` — kebab-case, must match a registered sensor in the site definition (Section 3).

- `modality` — one of `rf`, `acoustic`, `visual`, `radar`, `cooperative-traffic` (Section 11). Determines which `signature.*` sub-object is required.
- `position.accuracy_m` — 1-sigma horizontal error estimate. Used by the fusion layer for weighting.
- `confidence` — sensor-side belief this detection is a real object (not noise). Range [0.0, 1.0].
- `raw_reference` — optional pointer to the raw sample in Blob storage. Enables replay + forensic audit. Sensor is responsible for uploading the raw file BEFORE posting the detection metadata.
- `signature.{modality}` — modality-specific schema. See per-modality sub-sections below.

Signature sub-schemas

RF signature:

Field	Type	Notes
<code>band</code>	enum	Values from <code>threat_taxonomy.js RF_BANDS</code> . Complete list: 2.4-GHz, 5.8-GHz, 900-MHz, 433-MHz, UHF, L-band, S-band, C-band, X-band, Ku-band, Ka-band, HF, VHF, <code>silent</code>
<code>center_hz</code>	number	Center frequency in Hz
<code>bandwidth_hz</code>	number	Occupied bandwidth in Hz
<code>power_dbm</code>	number	Received signal power in dBm
<code>modulation_hint</code>	enum null	Best guess: <code>ofdm</code> , <code>fsk</code> , <code>qpsk</code> , <code>unknown</code> , or null when not determinable
<code>hopping_pattern</code>	object null	If frequency-hopping detected: { <code>pattern_hash</code> : string (hex, SHA-1 truncated to 16 chars), <code>dwell_ms</code> : number, <code>hop_count</code> : number }. Example: <pre>{"pattern_hash":"a3f9b2c1d8e7f4a5","dwell_ms":40,"hop_count":128}</pre>

Acoustic signature:

Field	Type	Notes
<code>profile</code>	enum	Values from <code>threat_taxonomy.js ACOUSTIC</code> (moped-buzz, high-whine, buzz, prop-hum, piston-drone, turbine-roar, jet-scream, rotor-thump, tilt-rotor, swarm-chorus, silent-electric, <code>unknown</code>)
<code>fundamental_hz</code>	number	Detected fundamental frequency
<code>harmonic_ratio</code>	number [0-1]	Strength of harmonic content vs noise floor
<code>spl_db</code>	number	Sound pressure level in dB

Visual signature:

Field	Type	Notes
<code>silhouette</code>	enum	Values from <code>threat_taxonomy.js VISUAL</code> . Complete list: <code>quadcopter-small</code> , <code>quadcopter-large</code> , <code>fpv-frame</code> , <code>delta-wing</code> , <code>swept-wing</code> , <code>straight-wing</code> , <code>boom-tail</code> , <code>v-tail</code> , <code>tandem-rotor</code> , <code>main-tail-rotor</code> , <code>coaxial-rotor</code> , <code>tilt-rotor</code> , <code>fixed-wing-general</code> , <code>cruise-missile</code> , <code>balloon</code> , <code>glider</code> , <code>unknown</code>
<code>bbox_px</code>	[x, y, w, h]	Detection bounding box in source image, pixels
<code>image_reference</code>	string	Blob URL pointing at the cropped detection thumbnail
<code>range_m</code>	number	Estimated range to target

Radar signature:

Radar sensors contribute primarily kinematic data + a coarse target class hint. Full classification is left to the fusion layer (Section 11) + NN classifier (Section 2) once radar tracks are paired with RF / acoustic / visual detections on the same target.

Field	Type	Notes
rcs_m2	number	Radar cross-section in square metres
doppler_ms	number	Radial velocity in m/s (positive = receding)
track_id	string	Radar's own internal track identifier (used for de-dupe across batches)
target_class_hint	enum	Coarse target category the radar's own processor infers: small-uav, medium-uav, large-uav, aircraft, helicopter, missile, bird, clutter, unknown
track_quality	number [0-1]	Radar's own belief this is a stable track (not noise / clutter)
first_seen	ISO 8601	Timestamp when the radar first acquired this track

Auth model

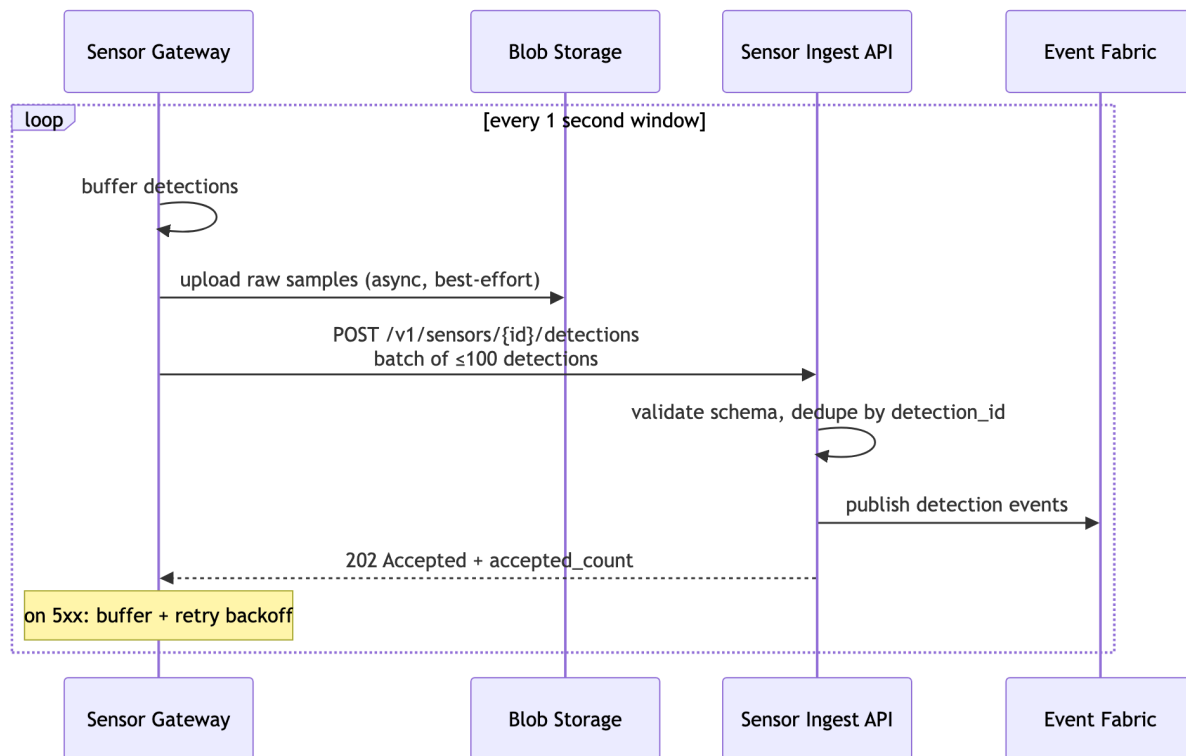
- OAuth 2.0 client credentials. Sensor gateway registers as a client with `sensor:write` scope + a tenant claim.
- Every request signed with `Bearer <access_token>` header.
- Access token TTL: 1 hour. Gateway refreshes 5 min before expiry.

Error semantics

- **400 Bad Request** — malformed payload, missing required field, coordinate out of range. Response `type` identifies which field failed.
- **401 Unauthorized** — token expired, invalid, or revoked.
- **403 Forbidden** — `sensor_id` not owned by the calling tenant.
- **409 Conflict** — `batch_id` already ingested (idempotency hit). Original response is echoed.
- **413 Payload Too Large** — batch exceeds 1 MB. Split and retry.
- **429 Too Many Requests** — rate limit. Retry after `Retry-After` seconds.
- **503 Service Unavailable** — server-side backpressure. Retry with exponential backoff (max 5 attempts).

Sensor gateway MUST NOT drop detections on 5xx. Buffer locally and retry until 2xx or the buffer's TTL expires.

Sequence diagram



Example

Curl request:

```
curl -X POST https://api.isr-systems.dk/v1/sensors/cph-rf-01/detections \
-H "Authorization: Bearer $TOKEN" \
-H "Content-Type: application/json" \
-H "X-Correlation-ID: 550e8400-e29b-41d4-a716-446655440000" \
-H "Idempotency-Key: batch-1726220000-001" \
-d @batch.json
```

Reference implementation

- **Mock ingest endpoint:** planned as `scripts/dev/mock-sensor-ingest.mjs` (stub — accepts any payload, logs to stdout, returns 202). Not yet in the repo.
- **Sensor gateway reference:** planned as a separate repo (`isr-sensor-gateway`) once wire format stabilises.
- **Sample batches for testing:** planned at `scripts/dev/sample-detections/*.json`. Not yet in the repo.

Open questions

- **Multi-modality per detection:** should a single detection carry both RF and acoustic signature slices when the sensor is a fused array? Current design says no (post as two detections, let fusion layer correlate). Revisit if bandwidth becomes an issue.
- **Backpressure signal:** currently 503 + Retry-After. Consider adding WebSocket push for immediate backpressure indication when API is under load.
- **Raw sample upload timing:** async best-effort today. Should we require raw upload confirmation before accepting detection metadata? Trade-off: latency vs forensic completeness.

2. NN classifier

Purpose

Raw detections from the sensor layer (Section 1) need to be classified — is this a DJI Mavic, a Shahed-136, or noise? The NN classifier contract specifies how a machine-learning service (running as an Azure ML endpoint or a self-hosted inference server) receives batched detections and emits classified events.

Actors

- **Caller:** ISR C2 event fabric (server-side, per-tenant instance)
- **Receiver:** NN classifier service (Azure ML endpoint, or self-hosted GPU node)

Wire format

POST `/v1/classify`

Request body:

```
{
  "classify_id": "cls-1726220000-abc",
  "context": {
    "site_id": "cph",
    "site_domain_scope": ["aviation", "ground"],
    "timestamp": "2026-09-13T10:30:00.520Z"
  },
  "detections": [
    {
      "detection_id": "det-1726220000-abc",
      "modality": "rf",
      "signature": {
        "rf": {
          "band": "2.4-GHz",
          "center_hz": 2437000000,
          "bandwidth_hz": 20000000,
          "power_dbm": -68.2,
          "modulation_hint": "ofdm"
        }
      },
      "position": { "lat": 55.617, "lon": 12.646, "alt_m": 120.5 }
    },
    {
      "detection_id": "det-1726220000-def",
      "modality": "acoustic",
      "signature": { "acoustic": { "profile": "moped-buzz", "fundamental_hz": 78.0, "spl_db": 54.3 } },
      "position": { "lat": 55.618, "lon": 12.647, "alt_m": null }
    }
  ]
}
```

Response 200 OK:

```
{
  "classify_id": "cls-1726220000-abc",
  "model_version": "isr-classifier-v0.3.2",
  "latency_ms": 87,
  "classifications": [
    {
      "detection_id": "det-1726220000-abc",
      "family": "commercial-quadcopter",
      "family_confidence": 0.82,
      "candidate_models": [
        { "id": "dji-mavic-3", "score": 0.51 },
        { "id": "dji-mavic-3-pro", "score": 0.28 },
        { "id": "dji-air-2s", "score": 0.11 }
      ],
      "classification": "unknown",
      "threat_estimate": "low",
      "rationale": "2.4-GHz + 5.8-GHz OFDM pattern consistent with DJI OcuSync 2.0 link. Model discriminator is inconclusive between Mavic 3 variants."
    },
    {
      "detection_id": "det-1726220000-def",
      "family": "loitering-munition",
      "family_confidence": 0.94,
      "candidate_models": [
        { "id": "shahed-136", "score": 0.71 },
        { "id": "shahed-131", "score": 0.19 }
      ],
      "classification": "hostile",
      "threat_estimate": "critical",
      "rationale": "Moped-buzz acoustic signature at 78Hz fundamental is a Shahed-class rotary engine tell. No cooperative flight plan for this position."
    }
  ]
}
```

Field constraints:

- `family` — MUST be a valid enum value from `threat_taxonomy.js` `FAMILIES` . Reject if the classifier can't map to a known family — return `unknown-signature` and let the routing matrix handle it.
- `family_confidence` — [0.0, 1.0]. Below 0.5 the routing matrix treats the family as `unknown-signature` regardless of the top pick.
- `candidate_models` — ordered by score descending. Model ids MUST exist in `threat_taxonomy.js` `MODELS` . Send at most 5 candidates.
- `classification` — enum: `hostile` , `friendly` , `unknown` , `resolved` . Classifier's initial guess; operator can override.
- `threat_estimate` — enum: `critical` , `high` , `medium` , `low` , `unknown` . Feeds the routing matrix's threat axis.
- `rationale` — one-line human-readable explanation. Surfaces in Agent B narrative + case-file audit.
- `model_version` — classifier's own version tag. Recorded on every classified event for later model-drift diagnosis.

Auth model

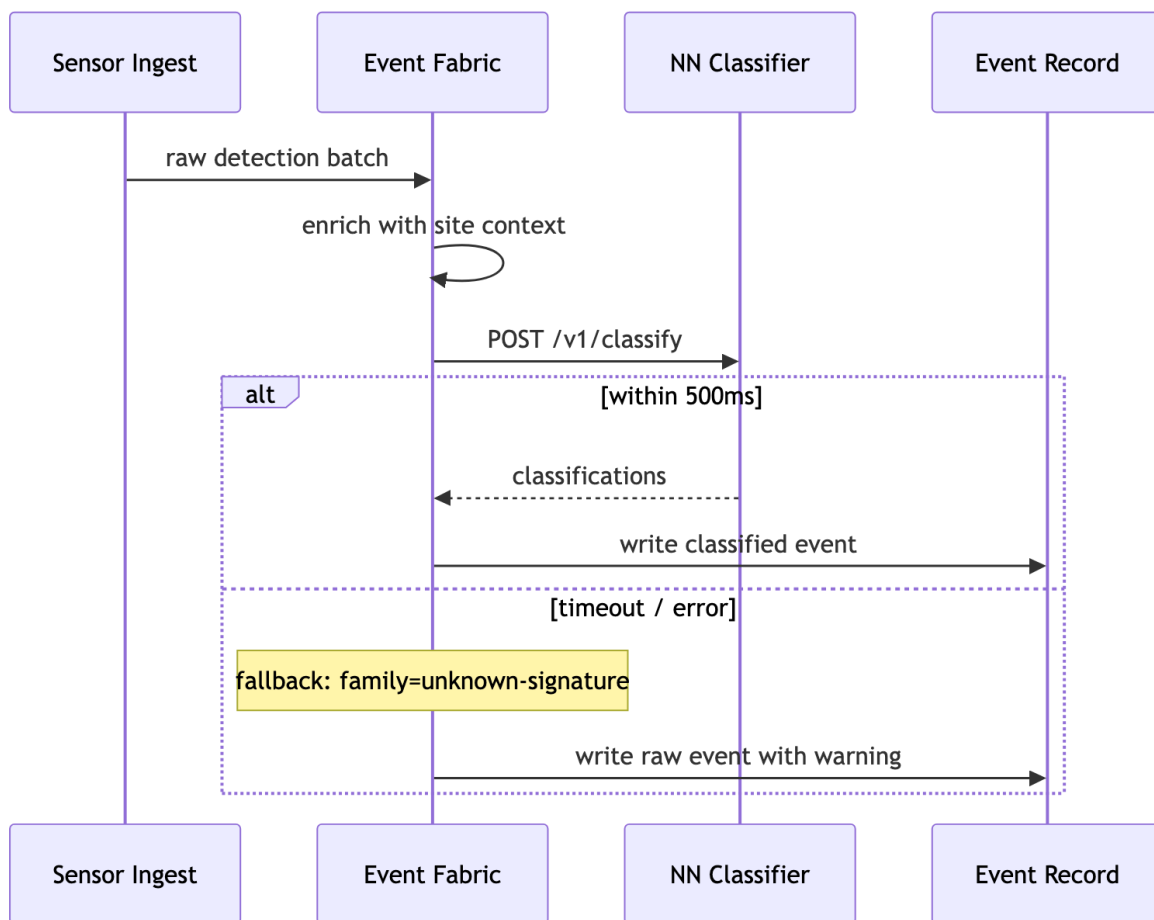
- Bidirectional mTLS between event fabric and classifier service.
- Classifier presents client cert; event fabric validates against a per-classifier trust anchor.
- Rotate certs every 90 days via Azure Key Vault.

Error semantics

- **400 Bad Request** — batch schema invalid.
- **408 Request Timeout** — classifier processing exceeded latency budget (default 500ms per batch). Event fabric proceeds with raw signature (family = `unknown-signature`).
- **422 Unprocessable Entity** — classifier produced output that fails the schema constraints above (e.g. invalid family value). Event fabric logs + falls back to `unknown-signature` .
- **500-503** — event fabric retries once with exponential backoff (100ms, 500ms). If both fail, detection proceeds with `unknown-signature` and a warning is written to the operator's Mission Console.

Latency budget: 500ms end-to-end per batch. Classifier services that can't meet this **MUST** be scaled horizontally, not slowed down. The event fabric will not wait past 500ms.

Sequence diagram



Example

Detection classified as Shahed → routing matrix (Section 9 of `docs/cross-agency-flows.md` "Threat taxonomy + routing matrix") fires strategic-strike + hostile-intel rules → auto-observers loop in.

Reference implementation

- **Mock classifier:** planned as `scripts/dev/mock-nn-classifier.mjs` — returns deterministic mappings for known signature slices (moped-buzz → shahed-136, high-whine + 2.4GHz → commercial-quadcopter, etc). Not yet in the repo; useful when landed for local testing without a real model.

- **Model versioning:** every classified event record stores `event.nn_model_version` (on `detection_subject.nn_meta.model_version` — see `events.js` schema) for later drift analysis. Separately, `event.narrativeCache.model_version` records Agent B's model version (Section 8).

Open questions

- **Batching vs streaming:** current design is request/response per batch. Consider WebSocket streaming for lower-latency use cases (e.g. tracking a fast-moving target across many frames).
- **Multi-modal classification:** should the classifier accept a single detection with multiple signature slices fused, or classify per-modality and let the platform reconcile? Current design: classifier gets per-detection input; platform-side fusion happens upstream in Section 11 (cooperative traffic fusion).
- **Explainability:** `rationale` is one line today. Should we add structured feature importance (which signature elements drove the classification)? Useful for regulator conversations.

3. Site definition

Purpose

Every customer site (airport, port, energy substation, data centre, government facility) is defined by a single YAML/JSON manifest that specifies boundaries, sensor placements, domain scope, tenant assignment, and default cascade recipients. The "7-step config-only add" claim rides on this schema.

Actors

- **Author:** ISR ops team (admin tenant) OR customer's own IT if they self-onboard
- **Consumer:** platform boot loader — reads all site manifests, registers each with `registerSiteDomains()` and the sensor + destination registries

Schema

`sites/{site_id}.yaml`

```
# Site identifier – kebab-case, unique across all tenants.
# Matches `event.siteId` throughout the codebase.
site_id: cph

# Human display label. Danish characters preserved.
label: Copenhagen Airport (Københavns Lufthavn)

# ICAO / IATA / port code – whatever the customer's operators call this
# site day-to-day. Optional but recommended for cross-reference.
code: CPH

# Tenant that owns this site. One of the OPERATORS in src/roles.js.
tenant: op-cph-airports

# Site type is DERIVED from domain_scope; kept as a hint for
# routing rules and doc generators.
site_type: airport

# Center point + boundary. Cesium consumes these for map rendering.
coordinates:
  lat: 55.618
  lon: 12.656
```

```
# Outer boundary as a closed GeoJSON-style polygon (lon, lat pairs).
# Coordinates in WGS 84 decimal degrees.
boundary:
  - [12.6301, 55.6284]
  - [12.6512, 55.6291]
  - [12.6598, 55.6202]
  - [12.6420, 55.6114]
  - [12.6303, 55.6183]
  - [12.6301, 55.6284]    # close the ring

# Which operational domains apply. Feeds event.domainScope and the
# threat_routing baseline rules. See DOMAIN enum in threat_routing.js.
domain_scope:
  - aviation
  - ground

# Sensors physically installed at this site. Each sensor is registered
# with the sensor ingestion contract (Section 1) at boot.
sensors:
  - sensor_id: cph-rf-01
    modality: rf
    position: { lat: 55.6181, lon: 12.6560, alt_m: 15 }
    coverage_radius_m: 3000
    label: "RF Spectrum Analyser (Terminal 3 rooftop)"
  - sensor_id: cph-acoustic-01
    modality: acoustic
    position: { lat: 55.6175, lon: 12.6540, alt_m: 8 }
    coverage_radius_m: 1200
    label: "Acoustic Array (Runway 22L)"
  - sensor_id: cph-radar-01
    modality: radar
    position: { lat: 55.6190, lon: 12.6600, alt_m: 45 }
    coverage_radius_m: 8000
    label: "Perimeter Radar (Control Tower)"
  - sensor_id: cph-eo-01
    modality: visual
    position: { lat: 55.6183, lon: 12.6555, alt_m: 25 }
    coverage_radius_m: 1500
    label: "EO/IR Camera (Terminal 3 pan-tilt-zoom)"

# Default cascade recipients when a hostile event occurs at this site.
# These are SUGGESTIONS pre-populated in the Mission Console; operator
# can accept, decline, or add more. Consumed by cascade_picker.js.
default_cascade_recipients:
  hostile_any:
    - pet
    - fe
    - politi-kbh
  hostile_high:
    - forsvarskmd
    - rigspoliti
    - beredskab
    - agency-traf

# Auto-observer role set that ALWAYS loops in on any event at this site,
```

```
# regardless of routing matrix. Empty by default; use sparingly.
always_observers:
  - agency-traf # Trafikstyrelsen is baseline for aviation domain anyway,
                # but naming here makes it explicit + audit-visible.

# Operating mode: 'live' (real sensors, operator-grade UX), 'sim'
# (cinematic drone POV for demos + training), or 'mixed' (per-detection
# source tag decides). Feeds the mode terminology per project memory.
mode: live

# Operator's dispatch scope – which asset kinds the operator's own team
# can dispatch. Empty array = SaaS-only tenant (no on-site assets).
# Referenced by _ROLE_DISPATCH_SCOPE_LOOKUP in main.js.
operator_dispatch_scope:
  - wildlife-response

# Airspace / regulatory context. Free-text notes surfaced in the
# case-file emphasis + Agent B narrative.
regulatory_context: |
  Class D controlled airspace up to 4500 ft AGL. Coordination with
  ATC on 118.100 MHz. NOTAM authority: Trafikstyrelsen. Standard
  drone incursion protocol: hold departures, delay arrivals >5 min,
  Trafikstyrelsen issues restriction NOTAM.

# Site-specific escalation SLA overrides (minutes). Defaults live in
# the escalation config; per-site overrides land here for sensitive
# sites. Empty means use platform defaults.
escalation_sla_minutes:
  hostile_high: 3 # override – tighter than default 5 min

# Reference to the per-site flow document that walks through the
# detection-to-closed-report lifecycle for this specific site.
flow_doc: docs/sites/cph-airport.md
```

Onboarding checklist

Adding a new site is a config-only change. The 7 steps:

1. **Create** `sites/{site_id}.yaml` using this schema.
2. **Add the tenant** to `src/roles.js` OPERATORS if it's a new customer (existing tenant: skip).
3. **Copy the site flow doc template** from `docs/sites/_template.md` to `docs/sites/{site_id}.md` and fill in.
4. **Register the sensors** in the sensor gateway config (per Section 1) so they know where to POST.
5. **Configure receiver adapter destinations** in `src/dispatch_source.js` if the customer has bespoke receiver APIs (per Section 5). Skip for SaaS-only tenants.
6. **Add site-specific test scenarios** to `scripts/dev/scenarios/` so simulation + regression tests cover the new site.
7. **Deploy**. The platform picks up the new site at next boot; no code changes.

Zero source files touched beyond the config manifest and (optionally) a receiver adapter registration for the new tenant.

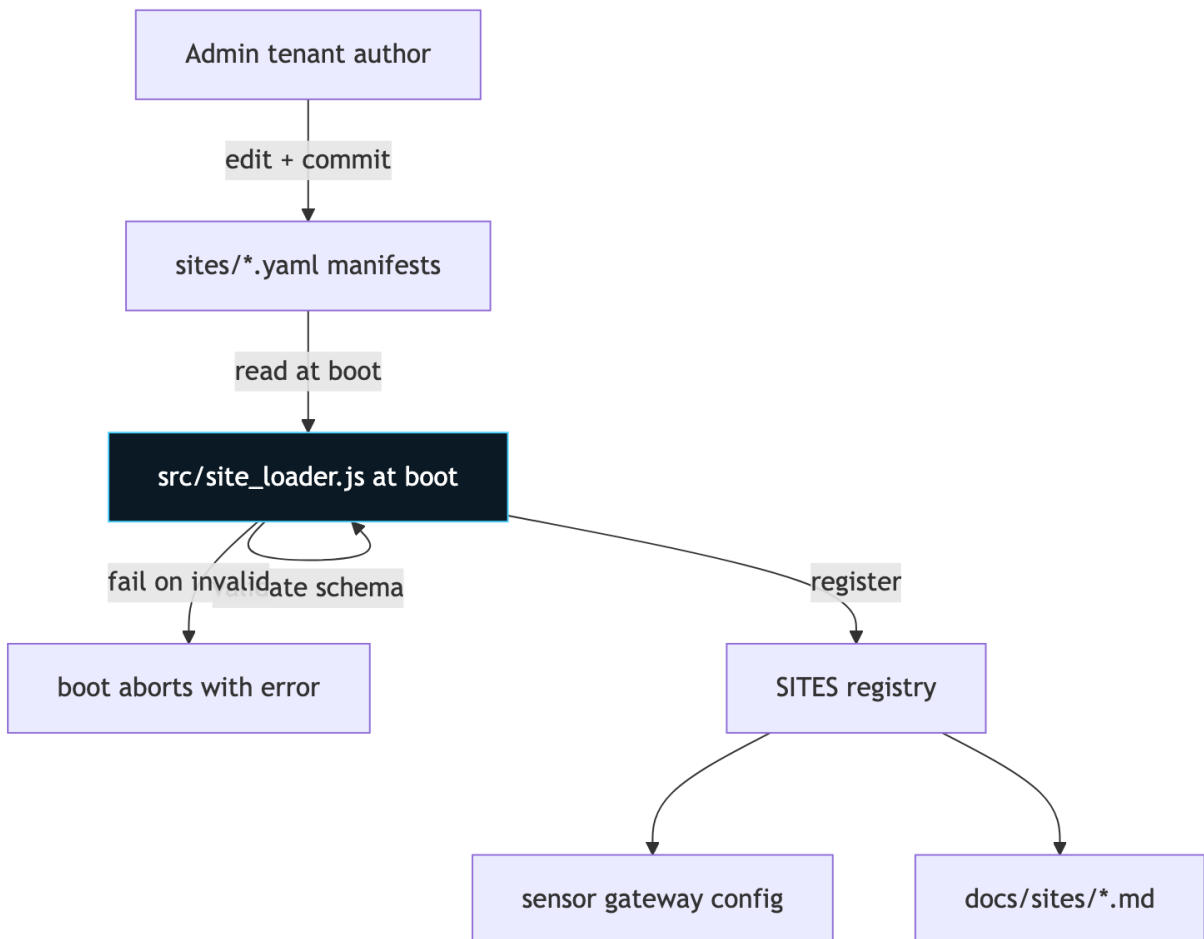
Validation

Every manifest is validated at boot by `src/site_loader.js` against a JSON Schema. Validation failures block boot with a clear error pointing at the manifest field + line number. Never ships a broken site to production.

Auth model

- **Read (any authenticated user):** site metadata is public within a tenant. Admin tenant sees all sites; operator tenants see only their own.
- **Write (admin tenant only):** site manifest changes require admin role. Audited to the feedback log (Section 10).

Sequence diagram



Reference implementation

- **Existing seed sites** in `src/sites.js` and `src/sites_energinet.js` — will be migrated to `sites/*.yaml` in a follow-up refactor.
- **Site loader:** `src/site_loader.js` (planned) — YAML parser + JSON schema validator + registry hookup.
- **Template:** `docs/sites/_template.md` — new-site flow doc scaffold.

Migration from existing `src/sites.js`

The manifest schema here is a **breaking change** from the current in-code SITES object. Field-name + casing differences that any transition tool or hand-migration must handle:

New manifest field	Old <code>src/sites.js</code> field	Notes
<code>site_id</code>	key of SITES object	Now explicit inside the manifest
<code>label</code>	<code>.name</code>	Renamed for consistency with role labels

boundary	.siteBoundary	Coordinate order preserved (lon, lat pairs)
sensors[].modality (single string)	.modalities: [] (array of TitleCase)	Split multi-modality sensors into separate manifest entries + lowercase
sensors[].coverage_radius_m	.coverageRadius (camelCase, implicit metres)	Renamed with explicit unit suffix
sensors[].position: {lat, lon, alt_m}	.position: [lat, lon] (2-tuple)	Now includes altitude explicitly

During the transition, `src/site_loader.js` will accept BOTH shapes and normalise to the new one internally. Deprecation window: 90 days from loader landing.

Open questions

- **Multi-site tenants:** does one operator tenant own multiple sites (e.g. CPH Airports A/S owns CPH + Roskilde)? Current design says yes — `tenant` field can repeat across manifests.
- **Site-of-site (sub-sites):** does an airport have sub-sites for terminal / runway / apron? Current design says no — treat as one site with sensor placement metadata distinguishing zones.
- **Dynamic re-registration:** can a site's sensor list change without a boot? Current design says restart required. Consider hot-reload for large fleets.

4. Post-Incident Report export

Purpose

Every closed event produces a Post-Incident Report (PIR) as a structured record. This contract specifies the export format when a receiver / operator / auditor needs to bundle one or more PIRs into a portable file (PDF for reading, JSON for machine consumption, CSV for spreadsheet analysis).

Actors

- **Caller:** authorised viewer in the platform UI OR API client with `pir:export` scope
- **Receiver:** PIR export service (Azure Function, per-tenant instance)

Wire format

GET `/v1/pir/{event_id}/export?format={json|pdf|csv}&scope={self|chain}`

Query parameters:

- `format` — `json` (structured record), `pdf` (rendered report), `csv` (flattened for spreadsheet analysis)
- `scope` — `self` (only this event) or `chain` (this event + every event in the same xlink chain per Section 7 of [cross-agency-flows.md](#))

Response headers:

```
Content-Type: application/json | application/pdf | text/csv
Content-Disposition: attachment; filename="PIR-{event_id}.{ext}"
X-Redactions-Applied: 2
X-Chain-Event-Count: 4
```

Detail is carried in the response body (`_redactions` field for JSON, watermark footer for PDF, dedicated column for CSV) — see Section 7 "Server-side visibility" for the header + body shape contract.

JSON export shape

```
{
  "export_version": 1,
  "generated_at": "2026-09-13T10:45:00.000Z",
  "generated_by": "receiver-tenant:politi-kbh",
  "correlation_id": "550e8400-e29b-41d4-a716-446655440000",
  "scope": "chain",
  "chain": {
    "chain_id": "chain-evt-1201",
    "size": 4,
    "first_at": "2026-09-13T09:12:00.000Z",
    "last_at": "2026-09-13T10:22:00.000Z",
    "span_minutes": 70,
    "sites": ["BILLUND", "CPH", "AALBORG"]
  },
  "events": [
    {
      "event_snapshot": {
        "id": "evt-1201",
        "site_id": "billund",
        "classification": "hostile",
        "threat": "high",
        "drone_type": "Shahed-136",
        "platform": "loitering-munition",
        "confidence": 0.94,
        "start_time": "2026-09-13T09:12:00.000Z",
        "end_time": "2026-09-13T09:47:00.000Z",
        "duration_sec": 2100,
        "outcome": "closed",
        "domain_scope": ["aviation", "ground"],
        "linked_event_ids": ["evt-1202", "evt-1203"]
      },
      "summary": "Shahed-136 signature detected at Billund airport perimeter...",
      "recommendation": "Trafikstyrelsen NOTAM restriction advised for next 30 min.",
      "chapters": [
        {
          "role_id": "politi-kbh",
          "role_label": "Politi København",
          "primary_archetype": "kinetic-response",
          "visibility_level_for_viewer": "full",
          "involvement": {
            "cascades_in": 1,
            "cascades_out": 0,
            "responses_sent": 1,
            "dispatches_owned": 2,
            "dispatches_open": 0,
            "catalog_entries": 3
          },
          "populated_archetypes": ["kinetic-response", "coordination-command", "public-safety-communication"],
          "timeline": [ /* ... per-role timeline slice ... */ ],
          "sub_sections": { /* ... per-archetype sub-section content ... */ }
        }
      ]
    }
  ]
  /* ... one chapter per contributor ... */
}
```

```

    ],
    "master_timeline": [ /* ... unified event timeline ... */ ],
    "escalations": [ /* ... per Section 6a of cross-agency-flows.md ... */ ],
    "dispatches": [ /* ... per Section 6a ... */ ],
    "catalog": {
      "subjects": [], "recordings": [], "resp_history": [],
      "attribution": [], "patterns": [], "xlinks": [], "roe": [],
      "evidence": [], "coord_decisions": [], "casualties": [],
      "advisories": [], "public_alerts": [], "liaison": []
    }
  }
  /* ... additional events in the chain when scope=chain ... */
],
"redactions_applied": [
  {
    "role_id": "pet",
    "reason": "INTEL chapter compartmented from viewer archetype 'kinetic-response'",
    "level": "summary"
  }
]
}

```

PDF export

Rendered from the JSON via a server-side templating engine. Each event becomes one section; each contributor chapter becomes a sub-section. Chain scope produces one PDF with an events index up front. Redacted chapters render as "Redacted for tenant boundary — contact authoring branch for access."

CSV export

Flattened one-row-per-detection-event view. Useful for spreadsheet analysis (average response time per receiver, dispatch outcome distributions, etc). Loses the per-contributor chapter nesting; use JSON if you need the full structure.

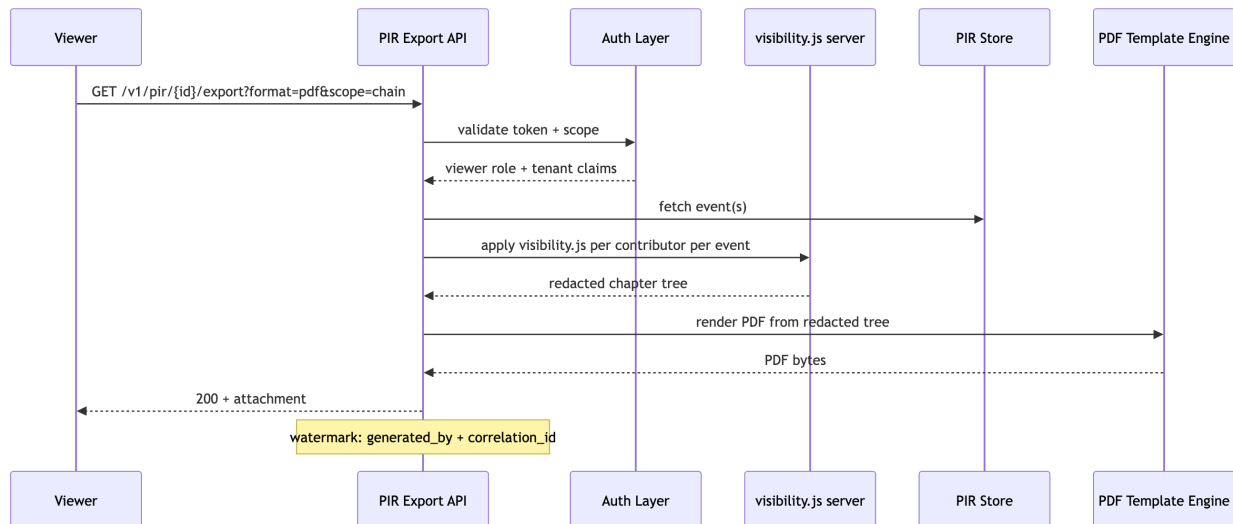
Auth model

- OAuth 2.0 token with `pir:export` scope.
- Viewer's tenant + role claims determine which chapters render at what visibility level. Server-side `visibility.js` gate (Section 7) applies before export streams.
- Exports are watermarked with `generated_by` + `correlation_id` for audit traceability.

Error semantics

- **404 Not Found** — event id doesn't exist OR viewer isn't cleared to see it at all (HIDDEN visibility for every chapter of the event).
- **403 Forbidden** — viewer lacks `pir:export` scope.
- **410 Gone** — event closed >6 months ago and archival tier is offline. Retrieval requires ops ticket.

Sequence diagram



Reference implementation

- **Client-side JSON download:** already exists — `data-rcv="pir-download"` button in the Step 7 panel. Serves the local `event.postIncidentReport` object.
- **Server-side export:** planned as Azure Function. Reads from Azure Blob PIR archive, applies visibility, streams response.

Open questions

- **Chain scope + partial visibility:** if the chain has 4 events but the viewer is cleared to see only 2, should the export drop the other 2 entirely, or include their stubs with "no visibility" markers? Current design: drop entirely + a footer noting "N additional events in this chain were not exported due to visibility policy."
- **Retention:** archival tier after 6 months? Legal + regulatory requirements TBD per customer.
- **Cross-tenant export:** can Politi Kbh export a chain that includes an event from Esbjerg Port? Only if the operator tenant explicitly grants cross-tenant read via the Auth+IAM contract (Section 12).

5. Receiver adapter

Purpose

The ISR C2 platform has a 386-agency receiver registry. When a cascade is sent, each recipient is notified through a per-role adapter that translates the platform's canonical escalation shape into whatever the receiving agency's real API expects. This contract specifies how any agency (Politi Kbh, BRS, Trafikstyrelsen, kommune SOC, hospital, NATO liaison, EU agency, kbr fire brigade, etc.) plugs a real API into the platform.

Actors

- **Caller:** ISR C2 escalation dispatcher (server-side, per-tenant)
- **Receiver:** custom adapter registered for a specific receiver role id (from `src/roles.js` RECEIVERS)

Wire format

Each receiver adapter implements 5 methods against a stable interface. Every method is idempotent (retries safe with the same `escalation_id` + version).

Adapter interface

Method names match the current `src/escalation_source.js` contract (lines 68-95). `sendEscalation` is REQUIRED; the other four are OPTIONAL — the platform calls them only if the adapter implements them.

```
interface ReceiverAdapter {
  /**
   * REQUIRED. Send a fresh cascade to this receiver. Called once per
   * new escalation. Adapter translates the assessmentPackage into
   * the receiver's real API shape.
   */
  sendEscalation(input: {
    escalation_id: string;           // stable id – same across retries
    event: EventSnapshot;           // minimal event context (see below)
    assessmentPackage: {
      operatorAssessment: string;
      agenticAssessment: { narrative, recommendation, model, generatedAt } | null;
      responseHistoryAtCascade: Array<DispatchSnapshot>;
      cascadeReason: string;
      priority: 'critical' | 'urgent' | 'standard';
      requesterRoleId: string;
    };
    initiatedBy: string;             // human display name of the sender
    initiatedAt: string;             // ISO 8601 UTC
    correlationId: string;           // propagates for tracing
  }): Promise<{
    receiverAcknowledged: boolean;   // did the receiver's system 200-ack
    receiverTicketId?: string;       // receiver's internal ticket if returned
    receiverMessage?: string;        // any human-readable response
  }>;

  /**
   * OPTIONAL. Withdraw a previously-sent escalation. Called when the
   * sender revokes the cascade before it's acted on. Idempotent:
   * repeat calls with the same escalation_id are no-ops.
   */
  withdrawEscalation?(input: {
    escalation_id: string;
    reason: string;
    withdrawnBy: string;
    withdrawnAt: string;
    correlationId: string;
  }): Promise<void>;

  /**
   * OPTIONAL. Update the sender's assessment on an existing escalation
   * without withdrawing it. Used when the situation evolves (e.g.
   * threat downgraded, new agentic take, additional dispatches fired).
   */
  updateAssessment?(input: {
    escalation_id: string;
    text: string;
    priority?: 'critical' | 'urgent' | 'standard';
    updatedBy: string;
    updatedAt: string;
    correlationId: string;
  }): Promise<void>;
}
```

```

/**
 * OPTIONAL. Send a threaded reply from the sender BACK to the
 * receiver – used when the operator responds inside the platform
 * to a receiver's earlier reply. Outbound call from platform to
 * receiver's system (not an inbound webhook).
 * Method name matches src/escalation_source.js.
 */
replyToEscalation?(input: {
  escalation_id: string;
  text: string;
  respondedBy: string;
  respondedAt: string;
  correlationId: string;
}): Promise<void>;

/**
 * OPTIONAL. Send a sender-side acknowledgement stamp to the receiver
 * (confirms "we saw your reply, we're on it"). Outbound. Method
 * name matches src/escalation_source.js.
 */
acknowledgeEscalation?(input: {
  escalation_id: string;
  acknowledgedBy: string;
  acknowledgedAt: string;
  correlationId: string;
}): Promise<void>;
}

```

Inbound webhook (receiver-initiated ack + reply): the current escalation flow does NOT include an inbound webhook path — receiver acknowledgements + replies today are recorded inside the platform via the operator's UI (Reply / Acknowledge CTAs). A future receiver-initiated webhook (their real system POSTs back to us) is on the roadmap; adapter interface will grow inbound callbacks then.

Registration

Every adapter registers per-role at boot in `src/escalation_source.js` :

```

import { registerEscalationAdapter, DEFAULT_ESCALATION_ADAPTER_KEY } from
'./escalation_source.js';
import { PolitiKbhAdapter } from './adapters/politi-kbh.js';

// Per-role registration.
registerEscalationAdapter('politi-kbh', new PolitiKbhAdapter({
  baseUrl: process.env.POLITI_KBH_API_URL,
  clientCert: process.env.POLITI_KBH_MTLS_CERT,
  // ... adapter-specific config
}));

```

Fall-through pattern. `getEscalationAdapter(roleId)` returns the per-role adapter if registered, otherwise the adapter registered against the sentinel key `DEFAULT_ESCALATION_ADAPTER_KEY` (`'__default__'`). The mock adapter (`src/adapters/escalation_mock.js`) registers itself against this sentinel at boot, so any unregistered role automatically gets the mock — logs to `event.interactions[]` + returns synthetic ack. Same sentinel pattern for dispatch (`DEFAULT_DISPATCH_ADAPTER_KEY` in `src/dispatch_source.js` , Section 6).

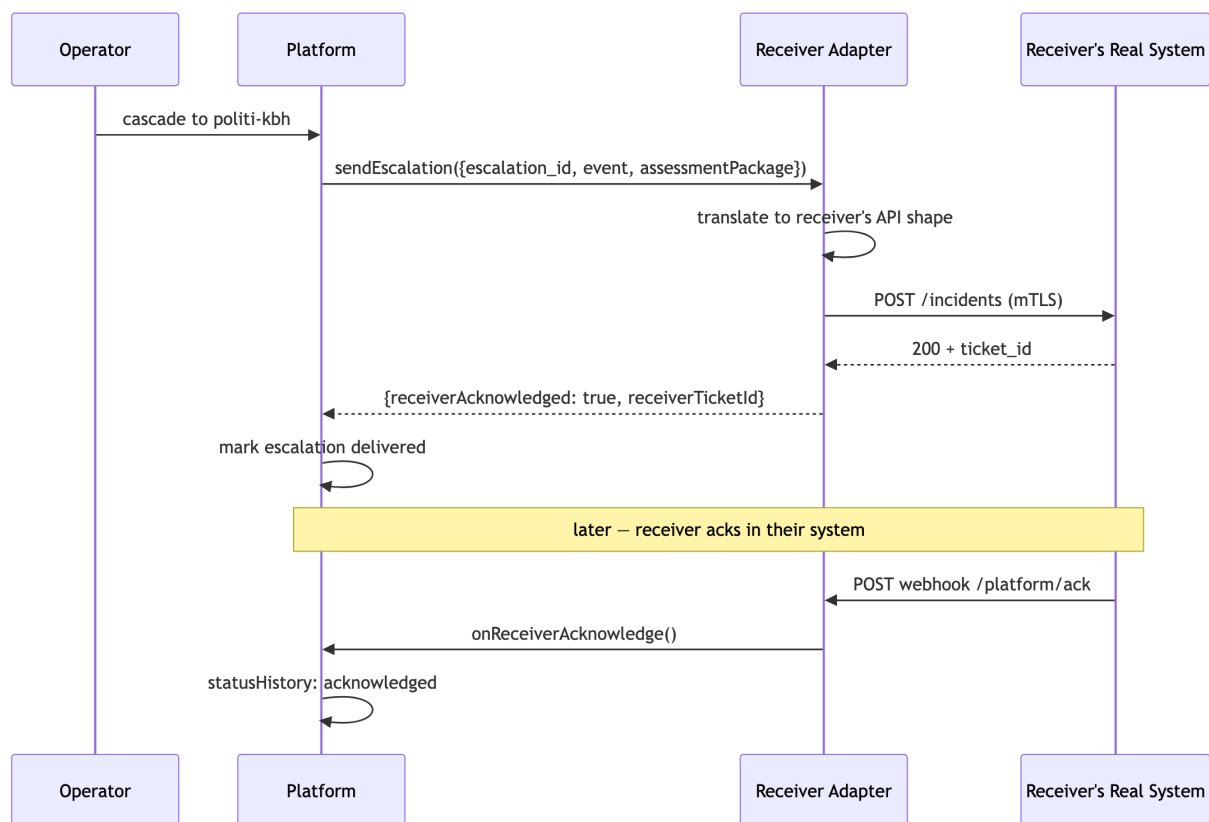
Auth model

- Adapter-specific. Some receivers use mTLS (defence + intel), some OAuth 2.0 client credentials (regulators + agencies), some IP-allowlisted API keys (legacy systems), some AS4 message-signed (EU agencies per eDelivery standard).
- Every adapter carries its own credential store. Platform never sees raw credentials — they live in Azure Key Vault, adapter reads at instantiation.

Error semantics

- Adapter methods throw on unrecoverable failure. Platform retries `sendEscalation` up to 3 times with exponential backoff (1s, 5s, 25s).
- After 3 failures, escalation is marked `blocked` with `blockedReason` set. Operator sees the block in Mission Console and can retry manually or route to a backup receiver.
- `withdrawEscalation` and `updateAssessment` are best-effort — failure logs a warning but doesn't block the platform (the underlying record is already in the audit trail).

Sequence diagram



Example implementation stub

```
// src/adapters/politi-kbh.js
export class PolitiKbhAdapter {
  constructor(config) {
    this.baseUrl = config.baseUrl;
    this.mtlsCert = config.clientCert;
    // ... init HTTP client with mTLS
  }

  async sendEscalation({ escalation_id, event, assessmentPackage, initiatedBy, correlationId }) {
```

```

// Translate ISR canonical shape → Politi Kbh's real POLIS incident format.
const politiIncident = {
  external_ref: escalation_id,
  site_code: event.siteId,
  severity: this._mapPriorityToPolisSeverity(assessmentPackage.priority),
  narrative: assessmentPackage.operatorAssessment,
  classification: this._mapClassification(event.classification, event.threat),
  requesting_officer: initiatedBy,
  // ... other Politi-specific fields
};
const response = await this._httpPost('/incidents', politiIncident, {
  headers: { 'X-Correlation-ID': correlationId },
});
return {
  receiverAcknowledged: response.status === 200,
  receiverTicketId: response.body?.polis_incident_id,
  receiverMessage: response.body?.desk_officer_notes,
};
}

async withdrawEscalation({ escalation_id, reason, correlationId }) {
  await this._httpPost(`/incidents/${escalation_id}/withdraw`, { reason },
    { headers: { 'X-Correlation-ID': correlationId } });
}

async updateAssessment({ escalation_id, text, priority, correlationId }) {
  await this._httpPatch(`/incidents/${escalation_id}`, {
    narrative_update: text,
    severity: priority ? this._mapPriorityToPolisSeverity(priority) : undefined,
  }, { headers: { 'X-Correlation-ID': correlationId } });
}

// Called by the webhook handler when Politi Kbh POSTs back.
onReceiverAcknowledge({ escalation_id, acknowledgedBy, acknowledgedAt, correlationId }) {
  // Delegated to platform's escalation lifecycle – this adapter just
  // forwards. Actual state mutation happens in events.js.
  this._platformCallbacks.onReceiverAck(escalation_id, {
    by: acknowledgedBy, at: acknowledgedAt, correlationId,
  });
}
}

```

Reference implementation

- **Mock adapter (default):** `src/adapters/escalation_mock.js` — accepts everything, logs to `event.interactions[]`, returns synthetic ack. Byte-for-byte matches pre-adapter behaviour.
- **Adapter registration:** `src/escalation_source.js` `registerEscalationAdapter(roleId, adapter)`
- **Per-adapter dir:** `src/adapters/*.js` — one file per real receiver adapter as they land

Open questions

- **Webhook receiver security:** every real receiver's webhook needs to authenticate BACK to the platform. Shared secret? Signed JWT? Per-adapter TBD.
- **Bulk withdraw:** if an entire chain is withdrawn (e.g. false-alarm event), do we call `withdrawEscalation` N times or add a batch endpoint? Current design: N calls. Batch is optional optimisation.

- **Adapter versioning:** if Politi Kbh changes their API, how do we deploy a new adapter version without downtime? Blue-green per adapter, or feature flag on the registration?
-

6. Dispatch adapter

Purpose

When a receiver dispatches a real asset (patrol car, helicopter, drone-counter unit, army C-UAS team, NOTAM issuance), the platform routes the command through a per-role dispatch adapter to the receiver's real asset-command API. This contract specifies the shape.

Detection-only invariant. Every dispatch in the platform routes through an operator's explicit Mission Console click — no code path today or in the current wire fires a dispatch without a human action. The Phase 3 target (three-method interface + signed operator confirmation JWT below) adds cryptographic enforcement at the wire layer for cross-tenant deployments. Phase 2 (today) enforces the invariant procedurally: every `adapter.dispatch()` call in `src/main.js` is behind an operator CTA + the feedback log audit trail (Section 10).

Actors

- **Caller:** ISR C2 dispatch dispatcher (server-side, per-tenant)
- **Receiver:** per-role dispatch adapter registered in `src/dispatch_source.js`
- **Downstream:** real customer asset-command API (Politi POLIS dispatch, BRS mobilisation, Trafikstyrelsen NOTAM issuance, etc.)

Wire format

Phase 2 (current — LANDED)

The dispatch adapter currently exposes a single method. Every real dispatch adapter today implements just this one:

```
interface DispatchAdapter_Phase2 {  
  /**  
   * Fire a dispatch to this receiver's real asset-command system.  
   * Method name matches src/dispatch_source.js current contract.  
   * Called synchronously from main.js after operator confirms in  
   * the Mission Console.  
   */  
  dispatch(input: {  
    action: string;           // dispatch action id from CD_PROFILE (e.g. 'police-c-uas',  
                              'helicopter-intercept')  
    event: EventSnapshot;     // event context  
    actorRole: string;        // role id of the receiver dispatching  
    actionDetail?: object;    // action-specific extras (asset selection, coords, etc.)  
  }): Promise<void>;  
}
```

Registration + fall-through same shape as receiver adapters (Section 5) — uses `DEFAULT_DISPATCH_ADAPTER_KEY` (`'__default__'`) sentinel. Mock at `src/adapters/dispatch_mock.js` preserves pre-adapter push shape byte-for-byte + writes to feedback log.

Phase 3 (planned — three-method interface with signed operator confirmation)

The full interface below is the target for cross-tenant production deployments where the platform must prove to a receiver's system that a human authorised the dispatch (not a rogue script or replay attack). Adds:

- A `fireDispatch` method with a signed `operator_confirmation` JWT payload
- An `onDispatchStateUpdate` inbound webhook (receiver's system pushes back progress)
- An `abortDispatch` method for in-flight recall

Migration from Phase 2 to Phase 3 is per-adapter (registered adapters can implement `dispatch()` OR `fireDispatch()` – the platform tries `fireDispatch` first, falls back to `dispatch`). Adapters that need cryptographic enforcement OR real-time state pushback move to Phase 3; simple mock + local test adapters stay on Phase 2 indefinitely.

Adapter interface

```
interface DispatchAdapter {
  /**
   * Fire a dispatch to this receiver's real asset-command system.
   * Called once per dispatch. Includes the signed operator
   * confirmation payload proving the human authorised the action.
   */
  fireDispatch(input: {
    dispatch_id: string;
    role_id: string;           // owner role (from RECEIVERS)
    dispatch_kind: string;     // from DISPATCH_KIND_ARCHETYPES in archetypes.js
    asset_id?: string;         // specific asset if selected (else adapter picks from pool)
    event: EventSnapshot;
    operator_confirmation: {
      operator_id: string;
      operator_role_id: string;
      signed_at: string;       // ISO 8601 UTC
      signature: string;       // JWT signed with operator's session key
    };
    correlationId: string;
  }): Promise<{
    accepted: boolean;
    dispatched_asset: { id, name, kind, eta_sec?, position? } | null;
    receiver_reference?: string; // receiver's own dispatch ticket
    rejection_reason?: string;
  }>;

  /**
   * Update dispatch state as it evolves. Called when the receiver's
   * system pushes back progress (e.g. asset en route, arrived,
   * engaging, complete, RTB).
   * Inbound webhook – adapter forwards to platform via callback.
   */
  onDispatchStateUpdate?(input: {
    dispatch_id: string;
    state: 'dispatched' | 'en-route' | 'arrived' | 'engaging' | 'complete' | 'rtb-started' | 'rtb-complete' | 'aborted';
    timestamp: string;
    metadata?: object;         // adapter-specific extras
    correlationId: string;
  }): void;

  /**
   * Attempt to abort an in-flight dispatch. Idempotent – repeat
   * calls with the same dispatch_id are no-ops.
   */
  abortDispatch(input: {
```

```

    dispatch_id: string;
    reason: string;
    requestedBy: string;
    correlationId: string;
  }): Promise<{
    aborted: boolean;           // false when the asset is past the abort window
    reason?: string;
  }>;
}

```

Registration

```

import { registerDispatchAdapter, DEFAULT_DISPATCH_ADAPTER_KEY } from './dispatch_source.js';
import { BRSAAdapter } from './adapters/brs.js';

// BRS Hedehusene is the beredskabscenter serving the greater Copenhagen /
// Sjælland region. Registration is per real role id from src/roles.js.
registerDispatchAdapter('brs-hedehusene', new BRSAAdapter({
  baseUrl: process.env.BRS_MOBILISATION_URL,
  apiKey: process.env.BRS_API_KEY,
  // ...
}));

```

Unregistered roles fall through to the `DEFAULT_DISPATCH_ADAPTER_KEY` sentinel — `src/adapters/dispatch_mock.js` registers there at boot, logs to console + feedback log + returns synthetic ack.

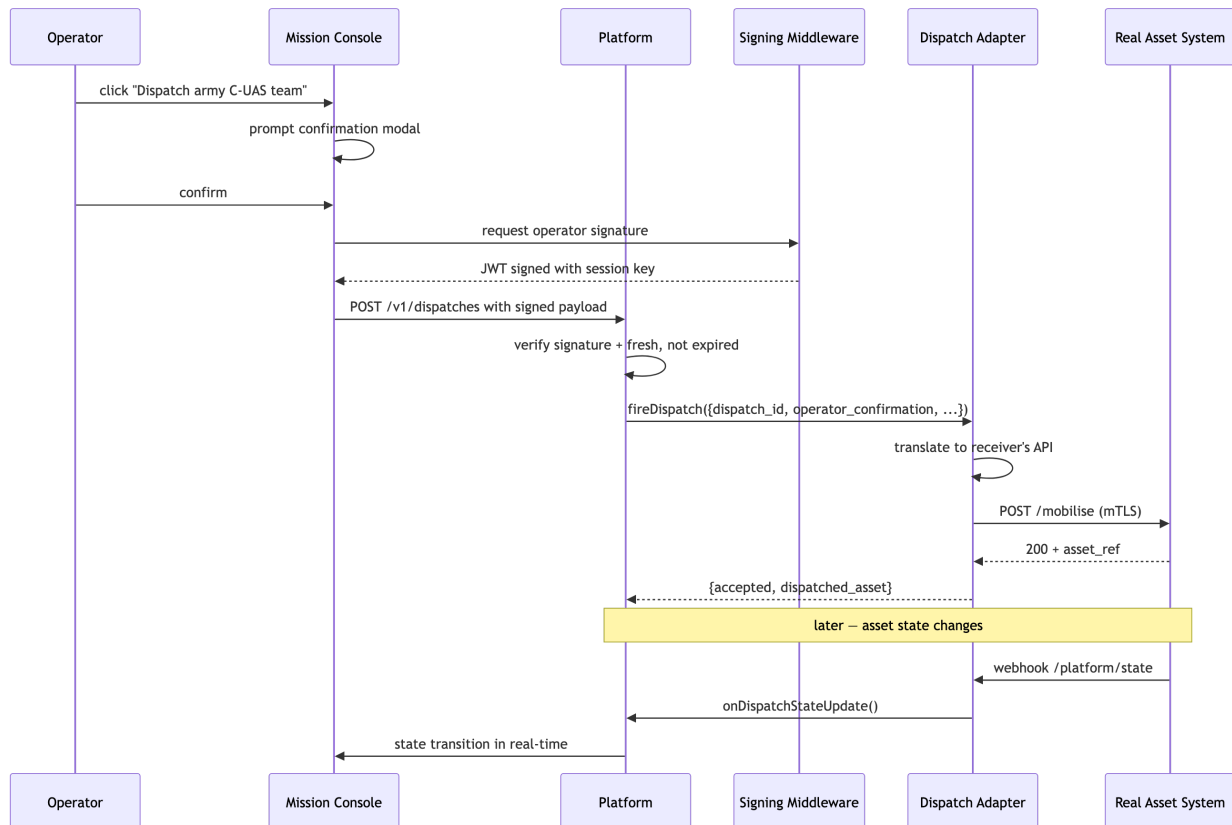
Auth model

- Adapter-specific (same as receiver adapters).
- Additionally, the `operator_confirmation.signature` is verified BEFORE the adapter is called. Platform middleware ensures no dispatch reaches an adapter without a valid, signed, unexpired operator confirmation. This is the detection-only invariant enforced at the wire.

Error semantics

- Adapter throws on unrecoverable transport failure. Platform retries once (1s backoff), then marks dispatch `failed` with reason.
- Adapter returns `{accepted: false, rejection_reason}` when the receiver's system explicitly rejects (asset unavailable, out of jurisdiction, wrong authority). Platform surfaces the rejection in Mission Console.
- `abortDispatch` returns `{aborted: false}` when the asset is past the abort window. Reason MUST be human-readable (e.g. "asset already engaged, cannot recall").

Sequence diagram



Reference implementation

- **Mock adapter (default):** `src/adapters/dispatch_mock.js` — preserves pre-adapter push shape byte-for-byte, logs to feedback log
- **Adapter registration:** `src/dispatch_source.js` `registerDispatchAdapter(roleId, adapter)`
- **Signature verification middleware:** planned as Azure Function pre-handler. Rejects any dispatch missing a valid operator confirmation JWT.

Open questions

- **Multi-asset dispatch:** if a rule dispatches N assets in one call (swarm engagement), do we call `fireDispatch` N times or extend interface to accept `assets: []` ? Current design: N calls (one adapter call per asset, easier to track independently).
- **Cross-role dispatch:** what if a dispatch spans multiple roles (joint operation)? Current design: split into N per-role dispatches, correlate via `event.id` + `parent_dispatch_id` .
- **Real-time asset telemetry:** should adapters stream position updates for in-flight assets, or is periodic webhook enough? Current design: webhook every state change; periodic position updates optional per adapter (Mission Console renders best-effort).

7. Server-side visibility

Purpose

`src/visibility.js` implements client-side visibility as defence-in-depth (INTEL/FORENSIC chapters redact to SUMMARY for non-cleared viewers, etc). The REAL access gate belongs at the API layer so a viewer with browser dev tools cannot bypass. This contract specifies the server-side visibility middleware — every read endpoint that returns event data passes through it.

Actors

- **Caller:** any authenticated user reading event data (PIR viewer, cascade picker, chain view, etc.)
- **Enforcer:** server-side visibility middleware (Azure Function pre-handler + per-route enforcement)

Wire format

The middleware is not a separate wire contract — it's a policy applied to every API response that contains event or chapter data. The BEHAVIOUR is what's contracted here.

Visibility levels

Applied per-chapter per-viewer. Same three-level model as `visibility.js` :

- **FULL** — every field renders as authored
- **SUMMARY** — nameplate + involvement counts only; situation, timeline, sub-sections replaced with redaction placeholder
- **HIDDEN** — chapter dropped from the response entirely; response includes `X-Redactions-Applied` header with a count

Rules

Server-side rules MUST mirror `src/visibility.js` rules exactly (see `chapterVisibilityFor()` and Section 7 "Visibility scoping" of `cross-agency-flows.md`). Divergence between client + server would produce split-brain redaction: viewer sees FULL client-side but the API refuses to serve the fields. Reconciled by:

- **Single source of truth:** the rule table lives in `src/visibility.js`. Server middleware imports the SAME module.
- **Compile-time check:** CI test asserts that server middleware and client renderer produce identical redaction output for a canonical corpus of (viewer, chapterRole, event) triples.

Response shape

Every response that carries event data includes:

```
X-Redactions-Applied: 2
X-Redactions-Detail: [
  {"role_id": "pet", "level": "summary", "reason": "INTEL compartmented from viewer archetype 'kinetic-response'"},
  {"role_id": "fe", "level": "summary", "reason": "INTEL compartmented from viewer archetype 'kinetic-response'"}
]
```

Redacted fields in the body are either:

- Replaced with `null` + `_redacted: true` marker at the field level, OR
- Chapter entirely omitted from an array (HIDDEN case)

Enforcement points

Every endpoint that returns event or chapter data MUST pass through the visibility middleware:

- `GET /v1/events/{event_id}` — full event with chapters
- `GET /v1/events/{event_id}/chapters/{role_id}` — single chapter
- `GET /v1/pir/{event_id}/export` — PIR export (Section 4)
- `GET /v1/chains/{chain_id}` — cross-event chain data
- `GET /v1/events?...>` — event list (per-summary + involvement chips only, redacted per viewer)

Endpoints that return NON-event data (e.g. `GET /v1/receivers` for the picker) don't go through this middleware — they have their own tenant-scoped filters.

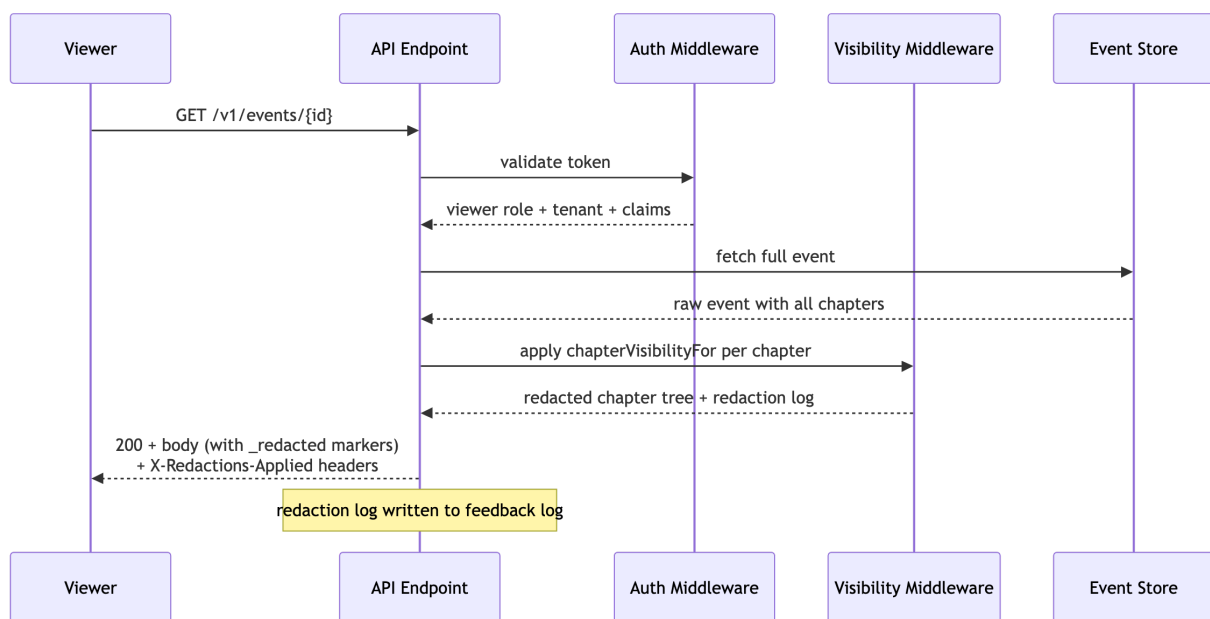
Auth model

- Middleware reads the viewer's tenant + role claims from the OAuth 2.0 access token (issued per Section 12).
- Admin bypass tokens are marked with a `bypass_visibility: true` claim. Only issued to Admin tenant sessions after explicit MFA re-auth for the read operation.

Error semantics

- **200 OK with redactions** — normal case, viewer sees what they're cleared for.
- **403 Forbidden** — viewer has no access to ANY chapter of the event. Response body contains a Problem Details explaining the tenant boundary.
- **404 Not Found** — event doesn't exist. Deliberately indistinguishable from 403 to avoid leaking event existence to unauthorised viewers.

Sequence diagram



Reference implementation

- **Client library:** `src/visibility.js` (LANDED, phase 6 of report shape)
- **Server middleware:** planned as Azure Function pre-handler `functions/middleware/apply-visibility.js` — imports the same `chapterVisibilityFor` function from `src/visibility.js` (shared source of truth).
- **CI parity test:** planned as `tests/visibility-parity.test.js` — generates canonical (viewer, chapterRole, event) triples + asserts client + server produce identical redaction output.

Open questions

- **Legal hold / eDiscovery:** does an authorised court order allow full-visibility retrieval bypassing normal rules? Yes, via admin bypass token + special audit trail. Formalise the workflow.
 - **Redaction proof:** should the server publish a cryptographic proof (hash chain) of redactions applied, so an auditor can verify the redaction pattern without seeing the redacted content? Consider for regulator conversations.
 - **Cache invalidation:** if a viewer's role changes mid-session (promotion, demotion, revocation), does their cached data get purged? Session refresh with role check every 5 min; hard revocation propagates via WebSocket push.
-

8. Agent B (Mistral)

Purpose

Agent B is the platform's narrative synthesis service. When an event closes (or on operator request during an active event), Agent B receives the event's classified data + site context + response history and returns a human-readable narrative + recommendation used in the Post-Incident Report + Mission Console + case-file. This contract specifies how the platform talks to the LLM backend.

Two-agent strategy: Agent A defines site context (offline, cached per site). Agent B correlates recorded event data against that context per detection.

Actors

- **Caller:** ISR C2 agentic layer (server-side, per-event trigger)
- **Receiver:** Mistral inference endpoint at Scaleway (default) or OVH (long-term production migration for SecNumCloud story)

Wire format

POST `/v1/agents/b/synthesize`

Request body:

```
{
  "synthesis_id": "syn-1726220000-abc",
  "event_id": "evt-1201",
  "site_context": {
    "site_id": "cph",
    "site_label": "Copenhagen Airport",
    "site_domain_scope": ["aviation", "ground"],
    "regulatory_context": "Class D controlled airspace up to 4500 ft...",
    "agent_a_cache_version": "cph-context-v3-2026-09-01"
  },
  "event_snapshot": {
    "id": "evt-1201",
    "classification": "hostile",
    "threat": "high",
    "drone_type": "Shahed-136",
    "platform": "loitering-munition",
    "confidence": 0.94,
    "start_time": "2026-09-13T09:12:00.000Z",
    "duration_sec": 2100,
    "outcome": "closed",
    "candidate_models": [
      { "id": "shahed-136", "score": 0.71 },
      { "id": "shahed-131", "score": 0.19 }
    ]
  },
  "response_history": [
    { "kind": "cascade", "to": "pet", "at": "2026-09-13T09:14:00Z", "priority": "urgent" },
    { "kind": "dispatch", "asset": "helicopter-intercept", "at": "2026-09-13T09:16:00Z", "state": "en-route" },
    { "kind": "cascade", "to": "beredskab", "at": "2026-09-13T09:18:00Z", "priority": "critical" }
  ],
}
```

```
"mode": "post-incident",
"language": "en",
"correlationId": "550e8400-e29b-41d4-a716-446655440000"
}
```

Response 200 OK:

```
{
  "synthesis_id": "syn-1726220000-abc",
  "model": "mistral-large-2411-scaleway",
  "generated_at": "2026-09-13T10:45:00.523Z",
  "latency_ms": 2340,
  "prompt_tokens": 1250,
  "completion_tokens": 380,
  "narrative": "A Shahed-136 loitering munition breached Copenhagen Airport airspace at 09:12 UTC from the southeast approach corridor. Signature confidence peaked at 94% within 3 minutes of first detection...",
  "recommendation": "Trafikstyrelsen NOTAM restriction advised for next 30 min while beredskab consequence teams sweep for debris. Consider pattern-of-life correlation with the September 2 Roskilde probe.",
  "narrative_language": "en"
}
```

Field constraints:

- `mode` — `post-incident` (called at `closeEvent`, narrative goes into PIR) or `live-briefing` (called during active event for Mission Console context).
- `language` — `en` or `da`. Danish version renders in the case-file when the viewer's tenant preference is Danish.
- `narrative` — 2-4 short paragraphs. Never contains raw model debug output, chain-of-thought, or system prompt leakage.
- `recommendation` — 1-3 sentences. Detection-only: recommends OBSERVATIONS or COORDINATION, never dispatch actions.
- `agent_a_cache_version` — pointer to the site-context cache Agent B correlated against. Recorded in the event so a later reader knows which context version informed the synthesis.

Auth model

- Mistral endpoint: bearer API key stored in Azure Key Vault, rotated 90 days.
- Client credentials: per-tenant. Rate limits per-tenant to prevent one noisy customer from starving others.

Error semantics

- 408 Request Timeout** — synthesis exceeded 5 seconds (post-incident) or 3 seconds (live-briefing). Platform falls back to the deterministic template in `post_incident_report.js` `_defaultSummary`.
- 429 Too Many Requests** — Mistral upstream rate limit hit. Platform queues the request with 60s backoff, retries once, then falls back to template.
- 502/503** — inference service unavailable. Platform falls back immediately + writes a warning to the event record. User sees "AI narrative unavailable — deterministic summary" in the case-file.

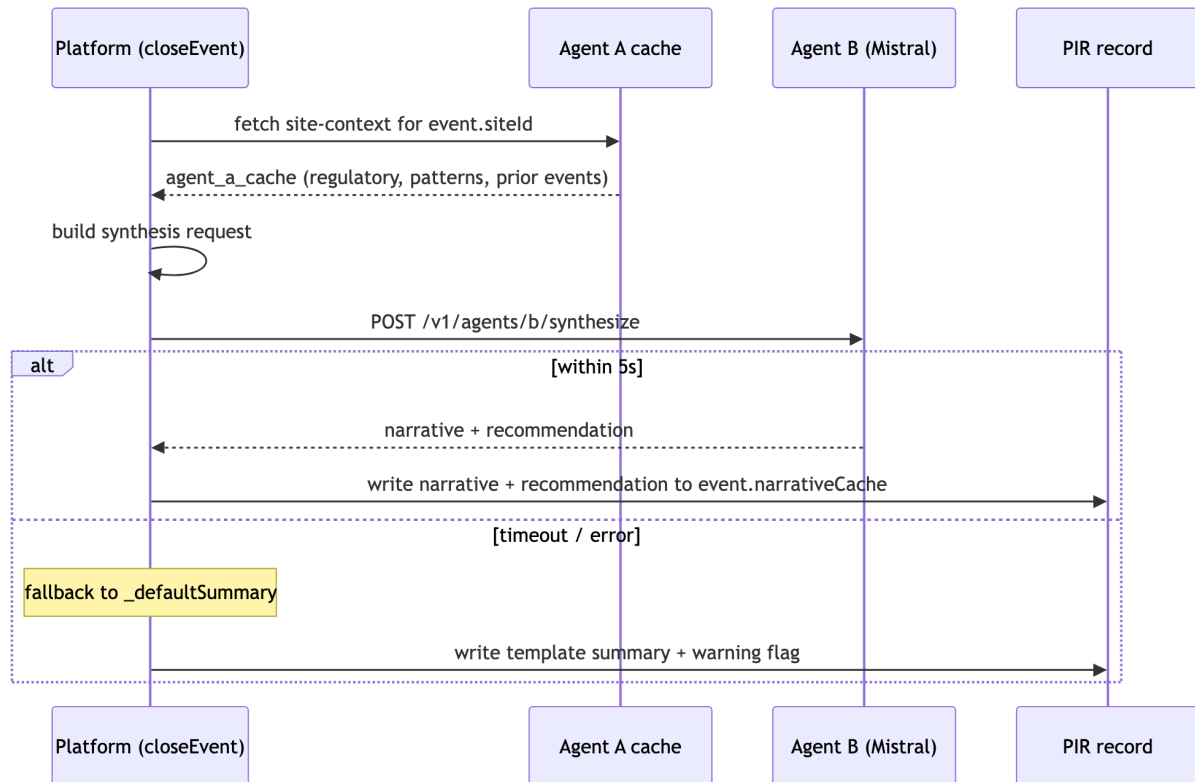
Detection-only invariant enforced at three layers:

- System prompt** (LANDED) — `DEBRIEF_WRITING_RULES_BASE` in `src/agents/agent_b_debrief.js` instructs Agent B never to recommend dispatch actions. Only observation / coordination / notification suggestions are permitted.
- Prompt-output validator** (planned) — regex + dispatch-verb dictionary that rejects any generation containing dispatch action verbs against a known list.

3. **Fallback (LANDED)** — on validator failure OR timeout OR error, fall back to `_defaultSummary` in `post_incident_report.js`.

Layers 1 and 3 are enforced today. Layer 2 is planned; until it lands, the invariant relies on the system prompt discipline + operator review of Agent B output in the case-file.

Sequence diagram



Reference implementation

Agent B is already LANDED across two per-mode modules on the platform side. The wire contract above describes the FUTURE hosted-service shape (Mistral endpoint reached over HTTP with the request/response schema); today the same call happens in-process via the internal streaming client:

- **Agent B post-incident (LANDED):** `src/agents/agent_b_debrief.js` — `streamDebriefNarrative` produces the narrative + recommendation at `closeEvent`. Includes `writeNarrativeCache` / `readNarrativeCache` for reuse in the PIR panel.
- **Agent B live-briefing (LANDED):** `src/agents/agent_case_file.js` — `streamCaseFileNarrative` for Mission Console live context. Imported at `main.js:570`, invoked at `main.js:20206`.
- **Transport (LANDED):** `src/mistral_client.js` — shared Mistral SDK wrapper used by both agents. Handles retry, backoff, streaming.
- **Deterministic fallback (LANDED):** `post_incident_report.js` `_defaultSummary` — template output when Agent B times out or errors.
- **Agent A cache spec:** `docs/agentic-signature-bridge-architecture.md` (existing, references two-agent strategy)
- **Prompt library:** planned as `src/agents/prompts/` — versioned prompt templates per (mode, language). Currently prompts live inline in each agent module (`DEBRIEF_WRITING_RULES_BASE` in `agent_b_debrief.js`, etc); factor out when the second language lands.
- **Hosted-service migration:** the wire contract above becomes real when Agent B moves from in-process to a hosted Mistral endpoint on Scaleway / OVH. Endpoint schema is what the migration writes against.

Open questions

- **Prompt versioning:** how do we handle a prompt template change mid-flight? Answer today: `agent_a_cache_version` + platform-side prompt version tag are both recorded on every synthesis. Migration = ship new prompt tag, old events retain their historical tag.
 - **Content filter tuning:** how strict is the "no dispatch recommendation" filter? Too strict → benign phrasing gets rejected. Too loose → LLM proposes kinetic action. TBD via corpus of golden test outputs.
 - **Danish language quality:** Mistral's Danish output quality? Fall back to English if quality gate fails? Operator can toggle per session.
 - **Model migration:** Mistral → OVH or SecNumCloud region for regulated deployments. Same wire contract, endpoint URL swap.
-

9. Signature bridge

Purpose

Raw NN classifier output (Section 2) emits family + candidate models. The signature bridge translates the sensor-side signature slice (RF band, acoustic profile, visual silhouette) into a family-labelled NARRATIVE that Agent B (Section 8) can reason over WITHOUT seeing raw numbers. Deterministic preprocessing, not a second LLM call.

This is the module that turns "78 Hz fundamental, 0.42 harmonic ratio, moped-buzz profile" into "distinctive Shahed-class rotary-engine acoustic signature." Every sensor observation gets a signature-bridge label so downstream consumers work in domain language, not signal-processing language.

Actors

- **Caller:** Event fabric (server-side, per-detection after NN classification)
- **Implementation:** pure JavaScript module — no external service call

Wire format

Not a network contract — a JavaScript API. Called synchronously on every detection during event ingest.

Input

```
interface SignatureBridgeInput {
  detection_id: string;
  modality: 'rf' | 'acoustic' | 'visual' | 'radar' | 'cooperative-traffic';
  raw_signature: RFSignature | AcousticSignature | VisualSignature | RadarSignature;
  nn_classification?: { // from NN classifier if it ran
    family: string;
    candidate_models: Array<{id: string, score: number}>;
    family_confidence: number;
  };
}
```

Output

```
interface SignatureBridgeOutput {
  family_label: string;           // e.g. "loitering-munition"
  family_confidence: number;      // [0.0, 1.0]
  candidate_models: Array<{
    id: string;                  // possibly refined from NN input via signature filter
                                // must exist in threat_taxonomy MODELS
```

```

    score: number;
    signature_match_reasoning: string;    // "RF: silent + L-band GPS; Acoustic: moped-buzz at
78Hz fundamental"
  }>;
  narrative_label: string;              // human-readable one-line description Agent B can
quote
  attribution_hints: Array<string>;      // e.g. ["origin:IR", "operator:RU", "kinetic-capable"]
  signature_hash: string;              // stable hash for precedent retrieval
}

```

Example transformation

Input:

```

{
  "detection_id": "det-1726220000-def",
  "modality": "acoustic",
  "raw_signature": {
    "profile": "moped-buzz",
    "fundamental_hz": 78.0,
    "harmonic_ratio": 0.42,
    "spl_db": 54.3
  },
  "nn_classification": {
    "family": "loitering-munition",
    "candidate_models": [
      { "id": "shahed-136", "score": 0.71 },
      { "id": "shahed-131", "score": 0.19 }
    ],
    "family_confidence": 0.94
  }
}

```

Output:

```

{
  "family_label": "loitering-munition",
  "family_confidence": 0.94,
  "candidate_models": [
    {
      "id": "shahed-136",
      "score": 0.71,
      "signature_match_reasoning": "Acoustic: moped-buzz at 78Hz fundamental (0.42 harmonic ratio)
is a Shahed rotary-engine tell. Not visible in signature: RF (autonomous per Shahed-136 profile)
or visual (out of E0 coverage).",
    },
    {
      "id": "shahed-131",
      "score": 0.19,
      "signature_match_reasoning": "Same acoustic signature as -136 but smaller airframe.
Distinguisher would require visual confirmation."
    }
  ],
  "narrative_label": "Distinctive Shahed-class rotary-engine acoustic signature detected.",
  "attribution_hints": ["origin:IR", "operator:RU-known", "kinetic-capable", "one-way-attack-

```

```
UAV"],
  "signature_hash": "sig-a3f9b2c1d8e7"
}
```

Rules-driven approach

Signature bridge uses a rules table (similar to the threat routing matrix). Each rule:

```
interface SignatureRule {
  tag: string;
  when: (input: SignatureBridgeInput) => boolean;
  narrative: string;           // human-readable label
  attribution: string[];       // hints
  model_filter?: (candidate: {id, score}) => boolean; // optional
}
```

Rules fire in order; multiple can match. Attributions dedupe. Narrative labels concatenate.

Example rules:

- `shahed-acoustic` — when `acoustic.profile === 'moped-buzz'` AND `70 <= fundamental_hz <= 90` → narrative "Distinctive Shahed-class rotary-engine acoustic signature detected." + attributions `[origin:IR, kinetic-capable, one-way-attack-UAV]`
- `dji-rf-ocusync` — when `rf.band` in `[2.4-GHz, 5.8-GHz]` AND `modulation_hint === 'ofdm'` → narrative "DJI OcuSync 2.0 link pattern." + attributions `[origin:CN, consumer-drone]`
- `stealth-signature` — when `rf.band === 'silent'` AND `visual.silhouette === 'delta-wing'` → narrative "Silent + delta-wing profile suggests loitering munition in autonomous cruise." + attributions `[kinetic-capable, autonomous]`

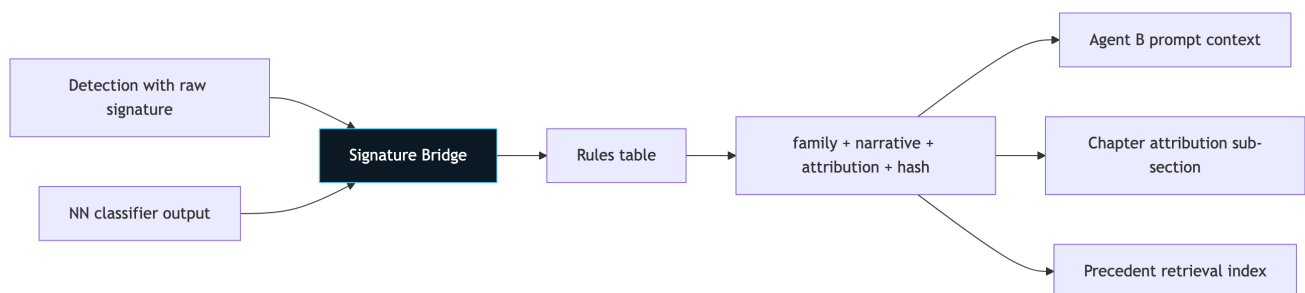
Auth model

Module-local. No external auth.

Error semantics

- Malformed input → returns `{family_label: 'unknown-signature', family_confidence: 0, narrative_label: 'Signature could not be parsed.'}`. Never throws.
- No rules match → returns `{family_label: nn_input.family || 'unknown-signature', narrative_label: 'Unrecognised signature pattern.'}`.
- Signature hash always computed even on error (stable hash over the raw signature bytes) — enables precedent retrieval regardless of classification.

Sequence diagram



Reference implementation

- **Existing spec:** docs/agentic-signature-bridge-architecture.md (LANDED, planning stage)
- **Module stub:** planned as src/signature_bridge.js
- **Rules table:** planned inside the same module, same shape as threat_routing.js RULES

Open questions

- **Sub-family discrimination:** how much can the bridge disambiguate Shahed-136 vs Shahed-131 from acoustics alone? Probably not; visual or RF gives the answer. Should the bridge downgrade candidate_models scores when only one modality supports the discrimination?
- **Attribution hint provenance:** should attribution hints carry a confidence + source-rule tag so an auditor can trace WHY the platform inferred "origin:IR"? Yes — add attribution: [{hint, confidence, source_rule_tag}] in v2 of the wire format.
- **Rule authoring workflow:** who writes new rules when a new adversary signature emerges (e.g. new Russian FPV variant)? Ops team or analysts? Track via signed rule additions to the rules table + audit log.

10. Feedback log WORM

Purpose

The feedback log is the platform's write-once audit trail. Every agent recommendation, every operator action, and every event outcome triple gets appended. Never read back into agent input (prevents Goodhart drift). Feeds regulatory audit + oncall retrospective + long-term learning post-incident.

Today: localStorage. Production: Azure Blob Storage with immutability policy (WORM). This contract specifies the Azure destination.

Actors

- **Writer:** platform server-side (Azure Function per-tenant instance)
- **Storage:** Azure Blob Storage container with Immutable Storage policy applied
- **Reader:** Admin tenant only, via authorised ops workflow. Never the operator or agent runtime.

Wire format

Write pattern

Every log entry is a single JSON object appended to a per-tenant, per-day append blob:

```
{
  "log_id": "fbl-1726220000-abc",
  "entry_type": "agent_recommendation",
  "timestamp": "2026-09-13T10:30:00.412Z",
  "tenant_id": "op-cph-airports",
  "correlation_id": "550e8400-e29b-41d4-a716-446655440000",
  "event_id": "evt-1201",
  "actor": {
    "role": "agent",
    "agent_name": "agent-b",
    "model_version": "mistral-large-2411-scaleway"
  },
  "recommendation": {
    "narrative": "Shahed-class signature detected at CPH...",
    "action_suggested": "notification-only",    // NEVER dispatch
    "confidence": 0.87
  }
},
```

```

"context_snapshot": {
  "event_classification": "hostile",
  "event_threat": "high",
  "site_id": "cph",
  "sensors_active": ["cph-rf-01", "cph-acoustic-01"]
}
}

```

Three entry types (the WRITE-ONLY triple):

1. `agent_recommendation` — every synthesis output from Agent B (Section 8) or signature bridge attribution (Section 9)
2. `operator_action` — every operator click that changes state (dispatch, cascade, respond, reclassify, close event, override AI recommendation, dismiss suggestion)
3. `event_outcome` — every terminal state (event closed with outcome X, dispatch completed with result Y, cascade acknowledged or overdue)

Each entry has a stable `log_id` (UUID) + timestamp + tenant + correlation_id for cross-reference. Never modified after write. Never deleted (retention: 10 years minimum, per Danish regulatory requirement).

Container layout

```

azure-blob-container: fbl-{tenant_id}-worm/
  daily/
    2026-09-13.jsonl          ← append blob, one JSON entry per line
    2026-09-12.jsonl
    ...
  monthly-summary/
    2026-09.json             ← auto-computed rollup (entry counts, hash chain root)

```

Immutability policy

- Azure Blob Immutability Policy: `Time-Based Retention Lock`, 10 years, non-editable.
- Once a policy is applied to a container, entries CAN NOT be modified or deleted for the retention period. Legal + regulatory guarantee.
- New append-only entries continue to write; old entries are locked.

Hash chain

Every daily append blob's tail includes a SHA-256 hash of the previous day's blob + the running content of the current day. Enables tamper detection: if an entry were somehow altered (e.g. by a compromised admin), the chain breaks + a nightly integrity job flags it. Chain root is published to a separate integrity ledger.

Auth model

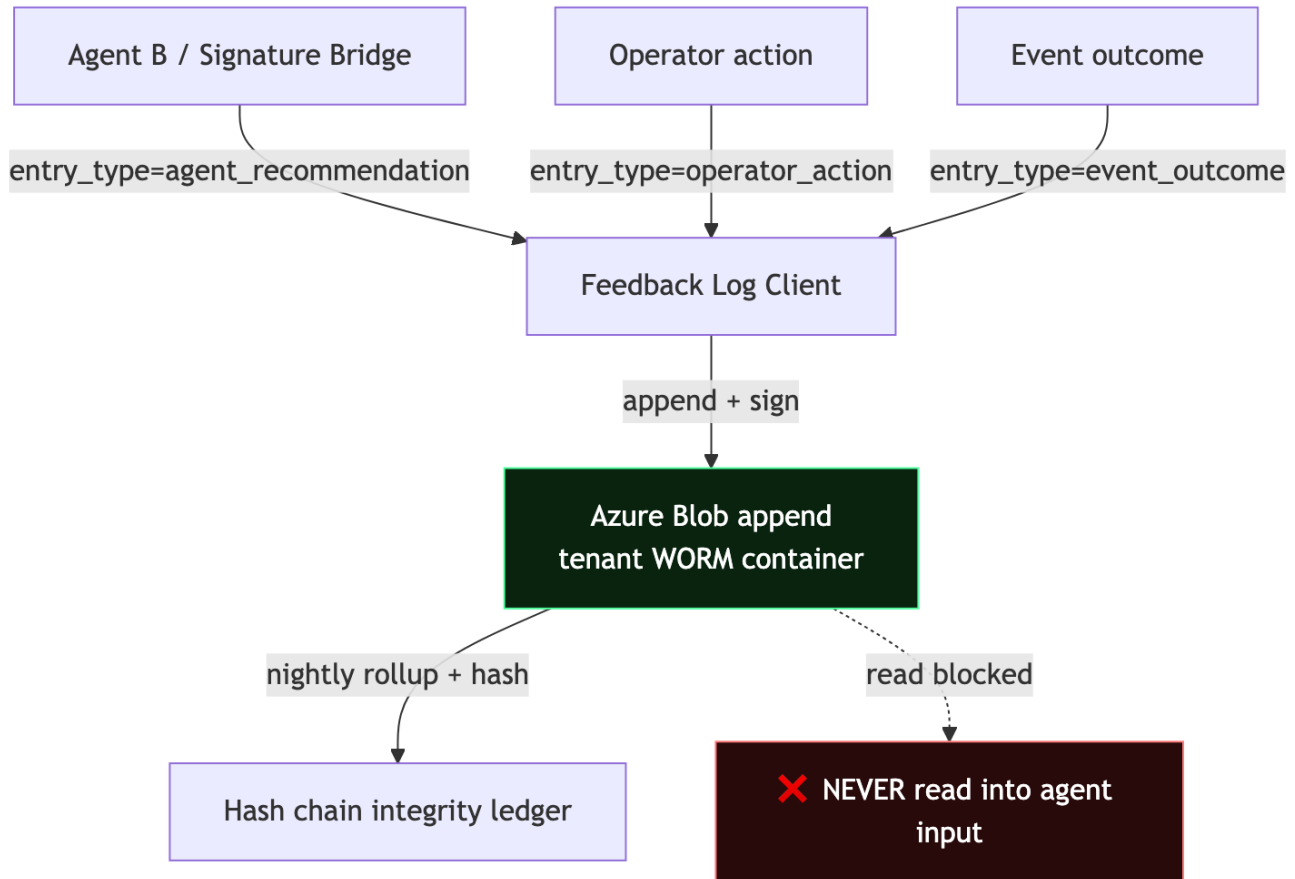
- **Write:** platform server-side identity (Azure Managed Identity per Function instance). Scoped to `Storage Blob Data Contributor` on the tenant's WORM container.
- **Read:** Admin tenant only. Requires MFA re-auth + explicit read-audit-log workflow. Every read logs to a SEPARATE meta-audit container (the "log of who read the log").
- **Never:** operator sessions, agent runtimes, or automated processes. There is no code path today or in the future that reads from this container into agent input. Enforced by Azure IAM + code review + CI test.

Error semantics

- **Write failure** — platform buffers in-memory (last 1000 entries) + retries with backoff. If buffer overflows, entries are marked LOST and logged to a separate degraded-mode log. Never blocks operator action.

- **Immutability policy violation** — Azure returns 409 Conflict. Platform logs + alerts. Cannot happen under normal operation (writes are always append).
- **Container full** — daily blob soft limit is 195 GB. Never hit under expected load (per-tenant entries << 1 GB/day even at 1000 events/day). If hit, auto-rotate to hourly blobs.

Sequence diagram



Reference implementation

The swap boundary is an adapter registry, not a single function. Real files:

- **Adapter registry (LANDED):** `src/feedback_log_store.js` — `registerFeedbackStore(key, adapter)` + `setActiveFeedbackStore(key)`. Every store implements the same interface; boot code picks one via the active key.
- **localStorage adapter (LANDED, dev default):** `src/adapters/feedback_log_localstorage.js` — implements the store interface via `localStorage` `_writeAll` + reads. Bounded size, auto-rotation when full.
- **Azure Blob WORM adapter (planned):** `src/adapters/feedback_log_azure_blob_worm.js` — implements the same interface against Azure Blob append + immutability policy. Registered at boot under a new key (`azure-blob-worm`); production tenants boot with `setActiveFeedbackStore('azure-blob-worm')`. Dev + local test keep the `localStorage` default.
- **Client-side write API (LANDED):** `src/feedback_log.js` — `writeAgentRecommendation`, `writeOperatorAction`, `writeEventOutcome`. Delegates to the active store via the registry.
- **Immutability policy setup:** planned doc `docs/ops/azure-blob-worm-setup.md` — Terraform + Azure IAM configuration.
- **Integrity job:** planned as an Azure Function running nightly, computing per-day hashes + publishing chain root.

Open questions

- **Cross-tenant meta-audit:** where does the "log of who read the log" itself live? Same tenant + immutable? Separate admin-tenant container? Currently planned separate + shorter retention (2 years) since it's meta.
 - **eDiscovery / court order:** authorised legal hold overrides immutability policy for retrieval (still no modification). Workflow TBD per Danish + EU legal requirements.
 - **Backup + geo-replication:** Azure GRS (geo-redundant storage) doubles the durability guarantee. Confirm cost fits customer contracts.
 - **Retention beyond 10 years:** some regulators (nuclear, aviation incident) require longer. Per-tenant retention override in the container policy.
-

11. Cooperative traffic fusion

Purpose

Every drone detection needs reconciliation against cooperative traffic (ADS-B for aircraft, Naviar for VFR + IFR flights in Danish airspace, similar sources for maritime). Without fusion, the platform would classify a legitimate general-aviation Cessna as unknown-signature.

This contract specifies how a cooperative-traffic feed plugs into the platform to enrich or explain-away detections.

Actors

- **Caller:** ISR C2 event fabric (server-side, per-site instance)
- **Receiver:** cooperative-traffic adapter (per-site) that talks to the real source (ADS-B receiver, Naviar API, marine AIS)

Wire format

GET `/v1/cooperative-traffic/{site_id}/active?lat={}&lon={}&radius_km={}×tamp={}`

Query parameters:

- Bounding box: lat / lon / radius_km
- timestamp: ISO 8601 UTC (usually near-real-time within 30s)

Response 200 OK:

```
{
  "site_id": "cph",
  "queried_at": "2026-09-13T10:30:00.000Z",
  "sources": ["ads-b", "naviair"],
  "tracks": [
    {
      "track_id": "adsb-4B0D19",
      "source": "ads-b",
      "callsign": "SAS1234",
      "icao_hex": "4B0D19",
      "position": {
        "lat": 55.6183,
        "lon": 12.6560,
        "alt_ft": 3500
      },
      "velocity": {
        "ground_speed_kts": 180,
        "heading_deg": 220,
        "vertical_speed_fpm": -800
      }
    }
  ]
}
```

```

    },
    "flight_plan": {
      "type": "ifr",
      "aircraft_type": "A320",
      "origin": "ESSA",
      "destination": "EKCH",
      "operator": "SAS"
    },
    "last_seen": "2026-09-13T10:29:58.000Z"
  },
  {
    "track_id": "naviair-vfr-DKPILOT01",
    "source": "naviair",
    "callsign": "OY-ABC",
    "position": {
      "lat": 55.6210,
      "lon": 12.6420,
      "alt_ft": 1200
    },
    "velocity": {
      "ground_speed_kts": 90,
      "heading_deg": 145,
      "vertical_speed_fpm": 0
    },
    "flight_plan": {
      "type": "vfr",
      "aircraft_type": "C172",
      "origin": "EKRK",
      "destination": null
    },
    "last_seen": "2026-09-13T10:29:52.000Z"
  }
]
}

```

Fusion logic (MVP · signal-block path)

The platform's event fabric queries this endpoint on every new detection. For each detection:

1. Query cooperative-traffic in a volume around the detection position + time window (defaults below).
2. Score each candidate cooperative track by (spatial distance × temporal proximity × track quality). Best score above threshold = match.
3. If a cooperative track matches: reconciler emits a `formatted_block` for Agent B's prompt context + records match metadata on the event.
4. If NO match: the detection stays classified per NN output + signature bridge. This is the "non-cooperative traffic" case — worth investigating.

Real matching defaults (from `src/cooperative_traffic_reconciler.js` header comments):

Parameter	Default
<code>match_radius_m</code>	800
<code>match_time_window_s</code>	4
<code>match_score_threshold</code>	0.75

Per-site tunable if a site's typical GPS accuracy differs.

Enrichment write-back (MVP shape)

When a match is found, the event record receives:

```
{
  "cooperative_match": {
    "match_count": 1,
    "matches": [
      {
        "track_id": "adsb-4B0D19",
        "callsign": "SAS1234",
        "score": 0.92,
        "matched_at": "2026-09-13T10:30:00.100Z"
      }
    ],
    "feed_status": "healthy",
    "formatted_block": "Cooperative traffic match: SAS1234 (A320, IFR EKCH-ESSA) at 3500 ft, ..."
  }
}
```

`formatted_block` gets injected into Agent B's context so the narrative can reference the cooperative track directly ("detection correlates with scheduled SAS1234 arrival").

Explicit MVP scope: auto-classification collapse (detection → friendly + threat=low) is OUT of MVP scope. Even a high-confidence cooperative match does NOT auto-downgrade the event's classification. Rationale: auto-classification would bypass the operator-override chain and undermine the detection-only stance (operator sees an already-classified event instead of an unclassified detection with cooperative evidence attached). Post-MVP: consider a "signal-block" path where the reconciler suppresses NEW event creation for cooperative-matched detections, but leaves existing events alone. See `cooperative_traffic_reconciler.js` header comment for design rationale.

Operator sees the `formatted_block` in the case-file + Agent B narrative + can manually reclassify to `friendly` after review. Every reclassification is written to the feedback log per Section 10.

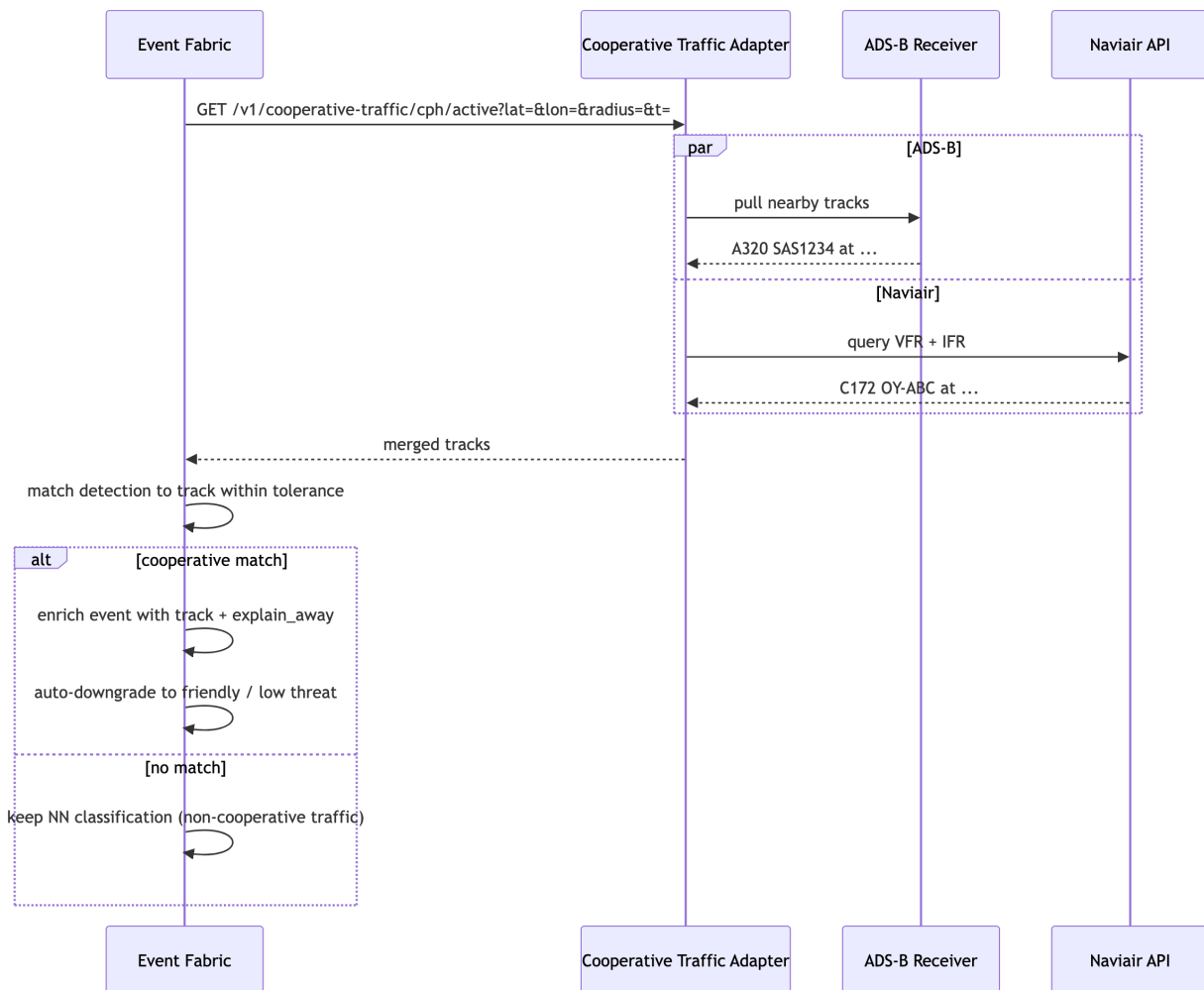
Auth model

- Adapter-specific. ADS-B receivers are usually LAN-local (no auth). Naviair API uses signed operator agreement + API key (Danish regulatory-approved credential store).
- Adapter caches responses for up to 5 seconds to avoid hammering the source on high-detection-rate scenarios.

Error semantics

- **Adapter unavailable** — platform proceeds without fusion. Detection stays classified per NN output. Event record notes `cooperative_match: null, fusion_status: "adapter-down"`.
- **Empty response (no traffic in area)** — normal outcome. Detection stays as-is; no `cooperative_match` field written.
- **Malformed response** — adapter logs + platform treats as no-response.
- **Latency budget** — 500ms. If adapter can't respond within budget, platform proceeds without fusion.

Sequence diagram



Reference implementation

- **Reconciler module (LANDED, MVP):** `src/cooperative_traffic_reconciler.js` — pure matching logic + formatted_block generator. Header comment explicitly scopes auto-classification-collapse out of MVP.
- **Existing spec:** `docs/agentive-cooperative-traffic-fusion-architecture.md` (planning + rationale)
- **Adapter stubs:** planned per source type — `src/adapters/cooperative-adsb.js`, `src/adapters/cooperative-naviair.js`, `src/adapters/cooperative-ais.js` for maritime. Each pulls from its real source + normalises into the response shape above.

Open questions

- **Tolerance tuning:** 150m horizontal is a starting guess. GPS accuracy of consumer sensors varies. Calibrate per-site.
- **Multi-track match:** what if two cooperative tracks fit the same detection? Rare but possible in busy airspace. Pick highest match confidence + log the ambiguity.
- **Historical playback:** cooperative-traffic sources publish historical data at different latencies. For replay + PIR, the platform pulls historical fusion at close time. Latency budgets for historical queries: 10s (best-effort).
- **Maritime:** AIS integration for port sites (Esbjerg) follows the same pattern but the adapter targets AIS receivers or Danish Maritime Authority's data feed. Separate adapter, same wire contract.

12. Auth + IAM

Purpose

Every actor in the platform (Admin, Operator tenant, Receiver tenant, sensor gateway, adapter service, agent service) authenticates through a common Identity + Access Management layer. This contract specifies tenant provisioning, session management, role assignment, revocation, and the claims model every other contract reads from.

Actors

- **Provider:** Azure AD B2C tenant `isr-c2` (production) / dev-tenant (staging)
- **Consumers:** every API endpoint in the platform, plus browser sessions for operator + receiver users

Tenant model

Three tenant types, matching `src/roles.js` ACCOUNTS:

Tenant type	Scope	Session lifetime	MFA
Admin (ISR)	Full platform access; provision + revoke other tenants; read audit logs	8h working session, 30 min inactivity timeout	Required (TOTP or FIDO2)
Operator (customer, e.g. CPH Airports)	Own site(s) + own events + cascade to receivers	12h working session, 60 min inactivity timeout	Required for dispatch confirmation
Receiver (agency, e.g. Politi Kbh)	Events they were cascaded to + own dispatches	12h session, 60 min inactivity	Required for dispatch confirmation

Claims model

Every OAuth 2.0 access token carries these claims. Example is a RECEIVER-tenant token (Politi Kbh officer session):

```
{
  "iss": "https://login.isr-systems.dk/tenants/isr-c2/v2.0",
  "sub": "user-abc-123",
  "aud": "https://api.isr-systems.dk",
  "exp": 1726263600,
  "iat": 1726220400,
  "tenant_id": "politi-kbh",
  "tenant_type": "receiver",
  "role_id": "politi-kbh",           // matches src/roles.js RECEIVERS id
  "role_archetype": "kinetic-response", // primary archetype from role's .archetype field
  "scopes": ["events:read", "cascades:respond", "dispatch:confirm", "pir:export"],
  "mfa_completed_at": "2026-09-13T08:00:00Z",
  "compartment_clearance": [],      // e.g. ["intel", "forensic"] for cleared reviewers
  "bypass_visibility": false,       // admin-only, requires explicit MFA re-auth
  "correlation_id_prefix": "politi-kbh"
}
```

An OPERATOR tenant token (CPH Airports session) carries `tenant_type: "operator"`, `tenant_id: "op-cph-airports"`, `role_id: "op-cph-airports"`, `role_archetype: "coordination-command"` (operators are coordination-role by default). Scopes differ: operators have `events:write`, `cascades:initiate`, `dispatch:confirm`.

Azure AD B2C claim naming: custom claims are declared as `extension_<name>` in the B2C custom policy XML (e.g. `extension_bypass_visibility`, `extension_compartment_clearance`) and rewritten to the token names above via a claims transformation. Saves an implementer a debug cycle when tracing why a claim didn't propagate.

Endpoints

POST `/v1/auth/token` (OAuth 2.0 token endpoint)

Standard OAuth 2.0 client credentials or authorization code flow. Returns access token + refresh token.

POST `/v1/auth/mfa/challenge`

For dispatch confirmation + admin bypass — issues a step-up MFA challenge. Response contains one-time challenge id; user completes challenge (TOTP or FIDO2); on success returns a short-lived (60s) `mfa_completed` token to sign the sensitive operation.

DELETE `/v1/auth/sessions/{session_id}` (Admin only)

Force-revoke a session. Propagates via WebSocket push to any active browser using the session; next API call from that session gets 401.

GET `/v1/auth/whoami`

Returns the current viewer's tenant + role + claims + effective visibility level. Used by the frontend to render the correct UI shell.

Provisioning workflow

1. Admin tenant creates a new tenant (Operator or Receiver) via Admin console.
2. Provisioning generates: OAuth 2.0 client credentials, initial admin user, tenant-scoped Azure Blob container (for feedback log), tenant-scoped API rate limits.
3. Admin adds initial users; sends email invitations with self-service password + MFA setup.
4. Each user's session token carries the tenant + role claims from that point.

Revocation reverses: DELETE tenant → invalidates all client credentials + sessions + closes API access + writes to admin audit trail.

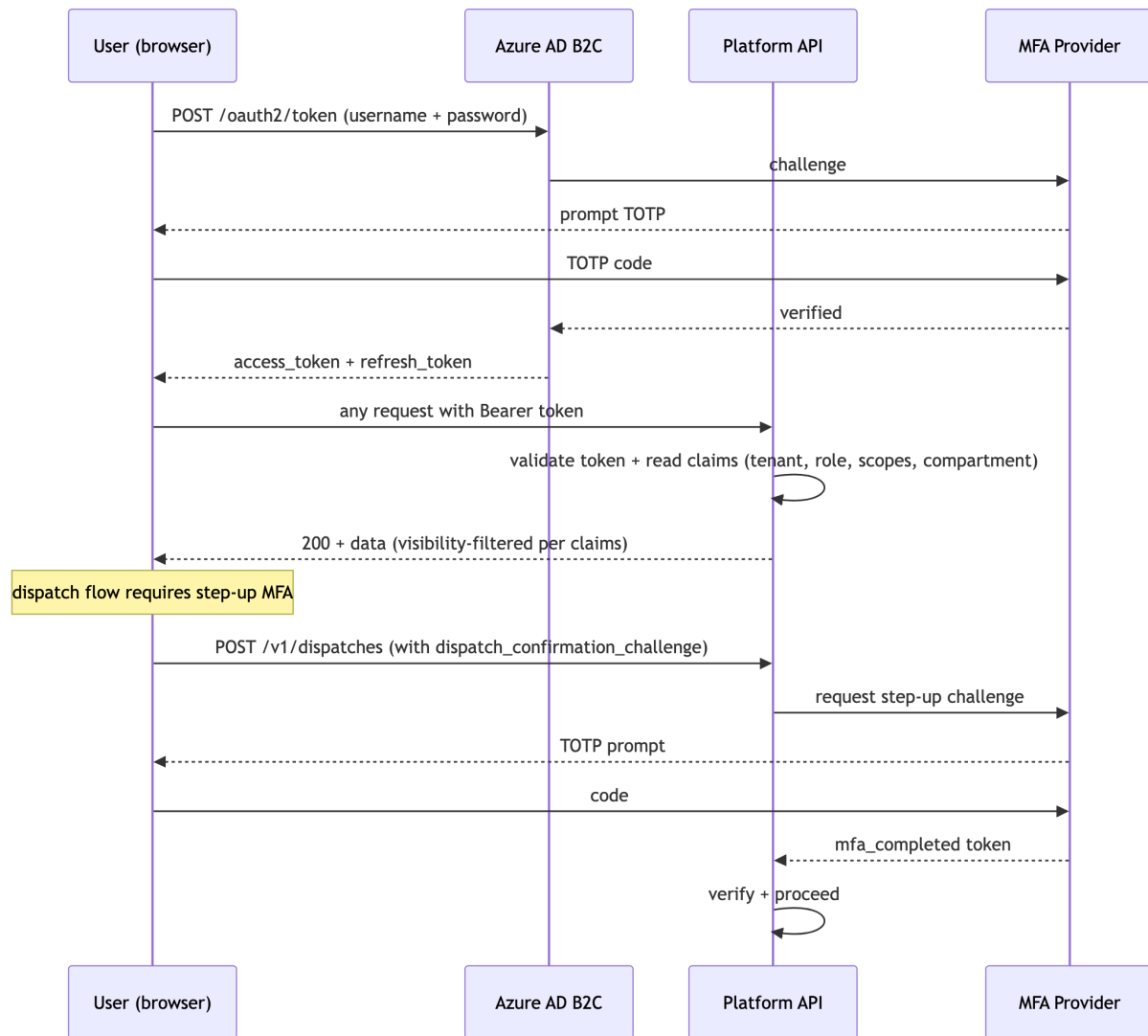
Auth model

Bootstrap credential: Azure AD B2C admin. All other credentials derived.

Error semantics

- **401 Unauthorized** — token expired, invalid, revoked. Client re-authenticates.
- **403 Forbidden** — token valid but scope insufficient for requested endpoint. Response includes which scope is required.
- **429 Too Many Requests** — brute-force auth attempts throttled per source IP.

Sequence diagram



Reference implementation

- **Client-side auth:** Azure MSAL library — planned integration into `src/auth/msal_client.js`
- **Server-side token validation:** JWT middleware — planned as Azure Function pre-handler
- **Admin console:** planned as separate app under `src/admin-console/`
- **Existing role registry:** `src/roles.js` — becomes the SOURCE OF TRUTH for role_id claims (server auto-syncs Azure AD B2C group memberships to role_id values)

Open questions

- **Multi-tenant single sign-on:** can an operator user seamlessly access a related receiver's view (e.g. if a customer owns both an airport AND is the local police coordinator)? Requires "multi-role sessions" — TBD.
- **Federated identity:** can a Politi Khb user log in with their own agency SSO (SAML federation)? Yes, planned via Azure AD B2C custom identity providers per receiver.
- **Break-glass admin:** if the primary admin loses MFA, how to recover? Escrowed recovery credentials in Azure Key Vault + notification to secondary admins.
- **Session sharing (kiosk mode):** operator control room with shared workstation, users badge in/out? TBD workflow.
- **Certificate-based auth for sensors:** covered under Sensor Ingestion (Section 1) mTLS. Certs issued via this Auth+IAM system.